NAVA: NEXT-GEN AGRICULTURAL VIRTUAL ASSISTANT

PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree of

MASTER OF SCIENCE IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

DHANUS V S
MG24C3135006

SREEGOVIND S
MG24C3135011

SCHOOL OF ARTIFICIAL INTELLIGENCE AND ROBOTICS
MAHATMA GANDHI UNIVERSITY
KOTTAYAM, KERALA, INDIA

JULY 2026

SCHOOL OF ARTIFICIAL INTELLIGENCE AND ROBOTICS

MAHATMA GANDHI UNIVERSITY
PRIYADARSINI HILLS P.O
KOTTAYAM, KERALA - 686560

CERTIFICATE

This is to certify that the project report entitled **"NAVA: Next-Gen Agricultural Virtual Assistant"** is a bonafide record of the successful work done by **Dhanus V S (Reg No: MG24C3135006)** and **Sreegovind S (Reg No: MG24C3135011)** in partial fulfillment of the requirements for the award of the degree of Master of Science in Artificial Intelligence and Machine Learning of Mahatma Gandhi University and that no part of this work has been submitted earlier for the award of any other degree.

Project Guides:

Dr. Deepa V
Assistant Professor (on contract)
School of Artificial Intelligence
and Robotics
Mahatma Gandhi University
Kottayam, Kerala

Dr. Mintu Movi
Assistant Professor (on contract)
School of Artificial Intelligence
and Robotics
Mahatma Gandhi University
Kottayam, Kerala

Honorary Director:

Prof. Dr. Bindu V R
School of Artificial Intelligence
and Robotics
Mahatma Gandhi University
Kottayam, Kerala

External Examiner:

Submitted for viva-voce on: . . . . . .

DECLARATION

We, **Dhanus V S** and **Sreegovind S**, hereby declare that the project report entitled **"NAVA: Next-Gen Agricultural Virtual Assistant"**, submitted to Mahatma Gandhi University, Kottayam in partial fulfillment of the requirements for the award of the degree of Master of Science (Artificial Intelligence and Machine Learning) is a record of the original work done by us during the period of study at the School of Artificial Intelligence and Robotics, Mahatma Gandhi University, Kottayam under the supervision and guidance of **Dr. Mintu Movi** and **Dr. Deepa V**, Assistant Professors (on contract), School of Artificial Intelligence and Robotics, Mahatma Gandhi University, and this project work has not formed the basis for the award of any degree of any University.

Place: Kottayam

Dhanus V S

Reg. No. MG24C3135006

Date: _____

Sreegovind S

Reg. No. MG24C3135011

Acknowledgement

We express our sincere gratitude for the opportunity to undertake and successfully complete our major project, ”NAVA: Next-Gen Agricultural Virtual Assistant.” This work would not have been possible without the guidance, encouragement, and support of many individuals.

We are deeply grateful to our internal guide, **Dr. Mintu Movi**, Assistant Professor (on contract), School of Artificial Intelligence and Robotics, for her constant motivation, invaluable guidance, and encouragement throughout this project. Her confidence in our abilities inspired us to continually improve and strive for excellence. We also extend our sincere thanks to our internal guide, **Dr. Deepa V**, Assistant Professor (on contract), School of Artificial Intelligence and Robotics, for her valuable guidance, continuous support, and constructive suggestions during the course of this work.

We sincerely thank all the faculty members and staff of the School of Artificial Intelligence and Robotics, Mahatma Gandhi University, for providing a supportive academic environment, valuable guidance, and constructive criticism throughout our postgraduate studies.

We are thankful to our friends for their encouragement, unwavering support, and companionship throughout this journey. We also express our heartfelt gratitude to our families for their unconditional love, patience, encouragement, and constant motivation.

Finally, we acknowledge the global scientific community whose research and innovations have inspired this work. We are grateful to the researchers, educators, and practitioners whose contributions have advanced the fields of artificial intelligence and agriculture. We also thank the countless open-source contributors and technical communities around the world whose software, frameworks, datasets, and shared knowledge made the development of this project possible.

We sincerely thank everyone who, directly or indirectly, contributed to the successful completion of this project.

Dhanus V S

Sreegovind S

Abstract

Crop diseases and delayed agronomic interventions cause significant reductions in agricultural yields, frequently affecting smallholder farmers who lack consistent access to expert guidance. Existing agricultural AI solutions often face challenges in real-world conditions due to reactive diagnosis and the tendency of Large Language Models (LLMs) to generate inaccurate chemical dosages or hallucinate advice. This project presents *NAVA (Next-Gen Agricultural Virtual Assistant)*, an end-to-end digital agronomist platform developed to provide localized, reliable agricultural assistance directly through a mobile-friendly web interface.

The proposed system employs a dual-model vision architecture. It utilizes an EfficientNet-B0 classifier for disease identification across multiple crops, achieving a validation accuracy of 94.54%. This is paired with *Thanal*, a virtual near-infrared (VNIR) estimation model that proactively identifies physiological plant stress from standard smartphone RGB images by evaluating scans against a rolling baseline.

To provide actionable and safe treatment recommendations, NAVA integrates a conversational agent powered by Llama-3 with a hybrid Retrieval-Augmented Generation (RAG) pipeline. The RAG system grounds the LLM's responses by dynamically routing queries, extracting key agronomic terms, and retrieving relevant guidelines from a verified database of institutional agricultural documents. Additionally, the platform incorporates Grad-CAM heatmaps for visual explainability, localized weather data injection, and a multi-level conversational memory that automatically tracks the farmer's actions. Experimental evaluations demonstrate that this integrated approach improves factual reliability compared to baseline parametric models, offering a practical, context-aware tool for proactive crop management.

Table of Contents

List of Figures

List of Tables

1 Introduction

Agriculture remains one of the most important sectors supporting global food security, economic stability, and rural livelihoods [1]. In developing regions such as India, and particularly in agricultural communities across Kerala, farming continues to be dominated by smallholder farmers who depend heavily on crop productivity for their income and sustenance [2]. However, modern agriculture faces numerous challenges including plant diseases, pest infestations, changing climatic conditions, nutrient deficiencies, and limited access to expert agronomic consultation. These challenges directly affect crop yield and quality, leading to significant economic losses for farmers [3].

Recent advances in Artificial Intelligence (AI), Machine Learning (ML), Computer Vision (CV), and Large Language Models (LLMs) have created opportunities for developing intelligent agricultural assistance systems capable of supporting farmers in decision making. Deep learning-based disease detection systems have demonstrated remarkable success in identifying plant diseases from images, while conversational AI systems have improved access to information through natural language interaction. Similarly, developments in explainable AI, retrieval-augmented generation, and precision agriculture technologies have enabled the creation of more transparent, reliable, and context-aware advisory platforms [4].

Despite these advancements, most existing agricultural solutions remain highly specialised and fragmented. Disease detection systems focus solely on classification, advisory chatbots often lack domain grounding, and monitoring systems frequently require specialised hardware that is inaccessible to small-scale farmers. Furthermore, many existing systems operate independently without retaining contextual information about farm history, crop conditions, or previous interactions [5].

To address these limitations, this project proposes **NAVA (Next-Gen Agricultural Virtual Assistant)**, a comprehensive multi-modal digital agronomist designed to provide disease diagnosis, physiological stress monitoring, explainable predictions, contextual agronomic recommendations, and persistent farm-level intelligence through a unified platform. By integrating computer vision, virtual spectral analysis, retrieval-augmented generation, contextual memory systems, and conversational AI, NAVA aims to bridge the gap between advanced agricultural expertise and practical field-level accessibility.

1.1 Motivation and Background

Agricultural productivity is significantly influenced by the ability to identify and respond to crop health issues at an early stage. Plant diseases alone contribute to substantial reductions in crop yield worldwide, affecting both food production and farmer profitability. Traditional disease diagnosis relies heavily on manual observation and expert consultation, processes that are often slow, subjective, and unavailable in many rural communities [6].

The rapid growth of smartphone accessibility among farmers has created an opportunity for AI-powered agricultural assistance systems. Computer vision techniques can analyse leaf images captured using ordinary smartphones, enabling automated disease identification without requiring specialised equipment. However, disease diagnosis alone addresses only part of the agricultural decision-making process. Farmers also require guidance regarding treatment options, preventive measures, environmental conditions, and long-term crop management strategies [7].

Large Language Models have emerged as powerful tools for natural language interaction and knowledge delivery. Nevertheless, generic LLMs are prone to hallucinations and may generate inaccurate agricultural recommendations if not properly grounded in verified sources. In agricultural contexts, such inaccuracies may result in incorrect pesticide application, financial losses, or environmental damage [8].

Another critical limitation of existing systems is their reactive nature. Most solutions detect diseases only after visible symptoms appear. By this stage, infections may already have spread extensively, reducing the effectiveness of corrective measures. Spectral analysis techniques, particularly near-infrared observations, have demonstrated the ability to identify physiological stress before visible symptoms emerge. However, traditional spectral imaging systems require expensive hardware that is impractical for widespread deployment among smallholder farmers [9].

These challenges motivated the development of NAVA as an integrated agricultural intelligence platform capable of combining disease diagnosis, proactive monitoring, contextual advisory support, explainable AI, and long-term farm memory within a single accessible solution.

1.2 Problem Statement

Existing agricultural support systems suffer from several significant limitations that reduce their practical effectiveness in real-world farming environments. Many disease detection models are trained using controlled laboratory datasets and exhibit poor generalisation when deployed under field conditions involving varying illumination, complex backgrounds, and diverse environmental factors. Furthermore, most diagnostic systems operate as isolated tools that provide predictions without offering contextual explanations, historical awareness, or actionable decision support.

Conversational AI systems introduce additional challenges. While modern language models can provide natural and intuitive interactions, they may generate inaccurate or unverified agricultural recommendations when operating without access to trusted knowledge sources. Such hallucinated responses pose serious risks when farmers rely on them for disease management, fertiliser application, or chemical treatment decisions.

Current agricultural monitoring approaches are also largely reactive, identifying problems only

after visible symptoms become apparent. Early-stage physiological stress, which often precedes observable disease symptoms, remains difficult to detect without specialised and costly sensing equipment. Consequently, farmers frequently miss opportunities for timely intervention.

Moreover, existing systems rarely maintain long-term awareness of farm activities, disease history, environmental conditions, and farmer decisions. The absence of contextual memory prevents the delivery of personalised and farm-specific recommendations, limiting the effectiveness of digital agricultural assistants.

Problem Statement:

To design and develop an integrated and context-aware multi-modal agricultural intelligence platform that provides proactive stress monitoring, interpretable disease diagnosis, and grounded agronomic recommendations through a unified digital agronomist interface.

1.3 Project Objectives

The primary objective of this project is to design and develop NAVA, a multi-modal digital agronomist capable of assisting farmers throughout the crop lifecycle using artificial intelligence and machine learning technologies.

The specific objectives of the project are as follows:

- To develop an image-based crop disease detection system capable of identifying multiple diseases across different crop categories.
- To incorporate Explainable Artificial Intelligence techniques for improving transparency and user trust in disease predictions.
- To implement a virtual near-infrared (VNIR) estimation framework for proactive monitoring of physiological crop stress using standard RGB images.
- To develop a conversational agricultural assistant capable of interacting with users through natural language and providing contextual agronomic guidance.
- To implement Retrieval-Augmented Generation techniques to ensure that agricultural recommendations are grounded in verified reference materials.
- To design a hierarchical memory mechanism capable of maintaining farm context, crop history, and user interactions across extended periods.
- To integrate disease diagnosis, stress monitoring, knowledge retrieval, environmental context, and conversational intelligence within a unified platform.
- To provide an accessible web-based interface that enables farmers to utilise advanced agricultural AI technologies without requiring specialised hardware.

1.4 Scope of the Project

The scope of NAVA encompasses the development of an integrated agricultural assistance platform that combines computer vision, machine learning, retrieval-augmented generation, and conversational artificial intelligence. The system is designed to support crop disease diagnosis, stress monitoring, contextual advisory generation, and farm management activities through a unified web-based environment.

The disease detection component focuses on the identification of multiple disease categories across selected crop species using deep learning-based image classification. Explainability mechanisms are incorporated to provide visual interpretations of model predictions. In addition, the system includes a VNIR estimation module that performs stress monitoring using RGB imagery, eliminating the need for dedicated spectral imaging hardware.

The conversational intelligence component provides natural language interaction capabilities while leveraging farm-specific context, historical records, environmental information, and retrieved agricultural knowledge. The retrieval subsystem ensures that responses remain grounded in verified agricultural reference materials rather than relying solely on generative model outputs.

Although NAVA demonstrates the feasibility of a comprehensive digital agronomist platform, certain aspects remain beyond the scope of the current implementation. These include multilingual deployment, large-scale field validation studies, native mobile applications, advanced satellite integration, and commercial-scale cloud infrastructure. Such enhancements are considered potential directions for future development and research.

1.5 Project Organisation

This report is organized into five chapters. Chapter 2 presents the literature review, discussing existing research related to crop disease detection, explainable computer vision, spectral analysis, agricultural large language models, and retrieval-augmented generation. Chapter 3 describes the methodology, including the system architecture and implementation of the major modules of NAVA. Chapter 4 presents the experimental results and analysis, covering both quantitative and qualitative evaluation of the developed system. Finally, Chapter 5 presents the conclusion, summarizes the work carried out, discusses its limitations, and outlines directions for future work. References and appendices are provided at the end of the report.

2 Literature Review

Agriculture continues to face significant challenges from crop diseases, environmental stresses, and limited access to expert agronomic guidance. Although deep learning and artificial intelligence have achieved remarkable success in agricultural applications, most existing solutions remain confined to isolated tasks such as disease classification, crop monitoring, or advisory generation. These systems often struggle with real-world deployment due to limitations in interpretability, contextual awareness, scalability, and resource efficiency. Furthermore, conventional disease detection approaches primarily rely on visible-spectrum imagery, making them ineffective for identifying physiological stress before visible symptoms emerge.

Recent advances in computer vision, hyperspectral imaging, large language models (LLMs), retrieval-augmented generation (RAG), and model optimization have created opportunities for developing intelligent agricultural assistants capable of supporting farmers throughout the crop lifecycle. These technologies facilitate accurate disease diagnosis, early stress detection, explainable decision-making, personalized advisory services, and efficient deployment on resource-constrained devices.

This chapter reviews the existing literature relevant to the development of the Next-gen Agricultural Virtual Assistant (NAVA). The discussion is organized into four major themes: Crop Disease Detection and Explainable Computer Vision, Spectral Analysis and Smart Farming Technologies, Agricultural LLMs and Intelligent Decision Support, and LLM Optimization and General AI Applications in Agriculture.

2.1 Crop Disease Detection and Explainable Computer Vision

Computer vision remains one of the most widely adopted technologies for automated crop disease diagnosis. Recent studies have focused on improving diagnostic accuracy while simultaneously enhancing model interpretability and practical usability.

Yan et al. (2025) proposed CDIP-ChatGLM3, a dual-model framework that integrates a computer vision model with a specialized agricultural language model. The study demonstrated that EfficientNet-B2 achieved an accuracy of 97.97% across multiple crops and diseases, while a fine-tuned ChatGLM3 model generated expert-level treatment recommendations. The research highlighted the effectiveness of modular vision-language architectures for agricultural applications, particularly in environments with limited computational resources. However, the framework relies primarily on disease labels rather than richer visual representations and lacks real-time knowledge retrieval capabilities [10].

Picon et al. (2022) addressed the limitations of conventional RGB imaging by introducing a deep learning framework that estimates a Virtual Near-Infrared (VNIR) channel from standard RGB images. Using a Dual-Head U-Net architecture, the model simultaneously reconstructed NIR in-

formation and performed vegetation damage segmentation. Experimental results demonstrated improved segmentation performance when VNIR information was incorporated, suggesting that hidden spectral information can enhance disease detection using inexpensive smartphone cameras. This work provides a foundation for early stress detection systems that do not require specialized multispectral sensors [11].

Devi et al. (2025) proposed HVAF-XAI-Net, a multimodal explainable AI framework for mulberry leaf disease detection. The framework combined Vision Transformers with Temporal Convolutional Networks to process both visual symptoms and environmental variables. Explainability techniques such as SHAP and LIME were incorporated to generate interpretable visualizations that highlighted the factors influencing model predictions. The study demonstrated that combining environmental context with image analysis significantly improves diagnostic performance while increasing user trust through transparent explanations [12].

Collectively, these studies demonstrate that modern disease detection systems are evolving beyond simple image classification toward multimodal, explainable, and context-aware frameworks. Their findings strongly support the development of NAVA's disease diagnosis and explainability modules.

2.2 Spectral Analysis and Smart Farming Technologies

The growing availability of advanced sensing technologies has enabled more sophisticated approaches to crop monitoring and early stress detection. Spectral analysis and virtual sensing techniques have emerged as promising solutions for identifying plant stress before visible symptoms appear.

Guerri et al. (2024) conducted a comprehensive review of deep learning techniques for hyperspectral image analysis in agriculture. The study highlighted the superiority of 3D Convolutional Neural Networks and hybrid CNN-Transformer architectures in extracting both spatial and spectral information from hyperspectral data. The review emphasized that hyperspectral imaging can reveal physiological changes, such as chlorophyll degradation and water stress, significantly earlier than traditional RGB-based approaches. However, the high cost and complexity of hyperspectral sensors remain major barriers to adoption among smallholder farmers [13].

Chourlias et al. (2025) introduced the concept of virtual sensors for smart farming. Rather than relying on extensive physical sensor networks, the proposed approach uses machine learning models to estimate environmental variables such as soil moisture and UV radiation from a limited number of available measurements. Experimental results showed that virtual sensors can achieve high predictive accuracy while reducing deployment costs. The study demonstrates how AI can democratize precision agriculture by providing farmers with access to valuable environmental information without requiring expensive infrastructure [14].

Chen et al. (2025) proposed a vision-language foundation model-based multimodal retrieval-

augmented generation framework for remote sensing applications. By combining visual representations with external knowledge retrieval, the framework significantly improved recognition performance in complex environments. The study demonstrated that retrieval-enhanced multimodal systems can effectively utilize external knowledge to compensate for limited training data and improve decision accuracy. Although developed for geological recognition, the underlying principles are highly relevant for agricultural monitoring and diagnosis [15].

These studies collectively highlight the importance of integrating spectral information, virtual sensing, and multimodal data processing into agricultural AI systems. Their findings provide strong theoretical support for NAVA's proposed VNIR-based early stress detection and contextual intelligence capabilities.

2.3 Agricultural LLMs and Intelligent Decision Support

Large Language Models have recently emerged as powerful tools for agricultural advisory systems. However, ensuring factual accuracy, domain-specific knowledge, and user trust remains a critical challenge.

Jiang et al. (2025) proposed K-ALLM, a knowledge-guided agricultural language model that integrates knowledge graphs with retrieval-augmented generation. The framework introduced self-supervised retrieval and knowledge-aware instruction tuning mechanisms to improve the handling of agricultural terminology and reduce hallucinations. Experimental results demonstrated superior performance compared to general-purpose language models, particularly in accurately generating recommendations involving pesticides, diseases, and crop management practices [16].

Naagarajan and Streif (2025) developed an interpretable AI framework for greenhouse management that combines natural language generation with advanced optimization-based control systems. The study demonstrated that translating complex control decisions into human-readable explanations significantly improves user understanding and trust. The findings highlight the value of conversational interfaces in bridging the gap between sophisticated AI systems and end users [17].

Deo et al. (2025) reviewed the scientific literature on crop planning at the farm level. The study identified key decision-making factors, including resource availability, environmental conditions, market dynamics, and sustainability objectives. The authors noted that many existing planning systems fail to incorporate real-time contextual information, limiting their practical usefulness. Their findings emphasize the need for intelligent advisory systems capable of integrating dynamic field conditions with long-term agricultural planning [18].

Together, these studies demonstrate the growing role of LLMs and intelligent decision-support systems in modern agriculture. Their contributions directly support NAVA's vision of providing context-aware, multilingual, and knowledge-grounded agronomic assistance.

2.4 LLM Optimization and General AI Applications in Agriculture

While large language models offer significant potential for agricultural advisory systems, their deployment in resource-constrained environments requires efficient optimization techniques. Additionally, broader reviews of AI applications in agriculture provide valuable insights into current trends and research gaps.

Yang et al. (2025) introduced EMWQ, an efficient mixed-precision weight quantization framework for large language models. By applying different quantization levels to sensitive and non-sensitive model components, the approach significantly reduced memory requirements while maintaining model accuracy. The study demonstrated that advanced quantization techniques can enable practical deployment of large models on devices with limited computational resources [19].

Wu et al. (2026) proposed KVC-Q, a dynamic quantization framework for compressing the key-value cache used during long-context inference in LLMs. The framework reduced memory consumption by approximately 70% while preserving performance across benchmark tasks. The proposed techniques are particularly relevant for conversational systems that require long-term memory and contextual awareness over extended interactions [20].

Attri et al. (2023) provided a comprehensive review of deep learning applications in agriculture, covering tasks such as disease detection, weed identification, and yield prediction. The review highlighted the dominance of CNN-based architectures while identifying persistent challenges related to interpretability, data scarcity, and poor generalization under real-world conditions. These findings reinforce the importance of explainability, diverse datasets, and lightweight architectures in practical agricultural AI systems [21].

Overall, these studies demonstrate that both efficient model optimization and robust deep learning methodologies are essential for translating agricultural AI research into deployable solutions capable of operating in real-world farming environments.

2.5 Summary

The reviewed literature demonstrates significant advancements across agricultural computer vision, spectral imaging, intelligent advisory systems, and efficient large language model deployment. Research in crop disease detection has increasingly emphasized multimodal analysis and explainability, while advances in spectral imaging and virtual sensing have opened new possibilities for early stress detection. Similarly, agricultural LLMs and retrieval-augmented frameworks have shown promise in delivering reliable, context-aware recommendations, and recent optimization techniques have made the deployment of sophisticated AI models on edge devices increasingly feasible.

Despite these advances, existing solutions typically address only isolated components of the

agricultural decision-making process. Few systems integrate disease detection, spectral analysis, environmental intelligence, explainable AI, knowledge-grounded advisory generation, and long-term conversational memory into a unified platform. NAVA seeks to bridge this gap by combining the strengths of these research directions into a comprehensive digital agronomist capable of supporting farmers throughout the crop lifecycle.

3 Methodology

This chapter describes the methodology adopted for the design and development of NAVA (Next-Gen Agricultural Virtual Assistant), a multi-modal agricultural intelligence platform that integrates computer vision, virtual spectral analysis, retrieval-augmented generation, and conversational artificial intelligence within a unified ecosystem. The overall objective of the system is to provide farmers with timely, reliable, and context-aware agricultural assistance through a single accessible interface.

The development methodology followed a modular architecture approach, enabling each functional component of the platform to operate independently while contributing to a shared decision-support workflow. The system was designed around five primary modules, namely Gathi, Mizhi, Mozhi, Yukthi, and the Shared Foundation Layer. Each module addresses a specific aspect of the agricultural assistance pipeline, ranging from user interaction and data management to disease diagnosis, stress monitoring, knowledge retrieval, and intelligent conversational support.

The following sections describe the overall architecture of NAVA, the datasets and preprocessing strategies employed, and the implementation details of each module that collectively enable the platform to function as a digital agronomist.

3.1 NAVA System Architecture Overview

NAVA was conceived as a comprehensive digital agronomist rather than a standalone disease detection system. To achieve this objective, the platform was designed as a collection of tightly integrated yet logically independent modules that collectively support agricultural monitoring, diagnosis, advisory generation, and farm management activities. This modular design improves maintainability, scalability, and extensibility while allowing individual subsystems to evolve independently without affecting the overall architecture.

At a high level, the platform consists of five major components:

- **Gathi** – The orchestration layer responsible for backend services, API management, user authentication, and frontend integration.
- **Mizhi** – The perception layer responsible for disease classification, explainable visual diagnosis, and VNIR-based crop stress monitoring.
- **Mozhi** – The conversational intelligence layer that manages dialogue generation, contextual reasoning, memory retention, and farm-aware interactions.
- **Yukthi** – The knowledge retrieval layer that provides retrieval-augmented generation capabilities using verified agricultural reference materials.

- **Shared Foundation Layer** – The infrastructure layer that provides persistent storage, configuration management, geospatial services, weather integration, and shared utilities.

The interaction between these modules enables NAVA to function as a unified agricultural decision-support platform. Users interact with the system through a web-based interface, where they can upload crop images, monitor plant health, manage farm information, and engage in natural language conversations. Requests initiated through the frontend are routed to the appropriate backend services by the orchestration layer, which coordinates communication between all system components.

Disease diagnosis and stress monitoring operations are handled by the Mizhi module. When a user uploads a crop image, the disease classification pipeline analyses the image and predicts the most probable disease category. Simultaneously, the VNIR estimation subsystem evaluates physiological stress indicators by generating virtual near-infrared representations from standard RGB imagery. The results of both analyses are stored within the farm database and become part of the contextual information available to other system components.

Conversational interactions are managed by Mozhi. Rather than functioning as a conventional chatbot, Mozhi constructs responses using multiple contextual sources including farm metadata, crop history, disease records, VNIR monitoring results, weather conditions, and previous conversations. This contextual information enables the system to provide personalised responses that are specific to an individual farm rather than generic agricultural recommendations.

To improve factual reliability, Mozhi collaborates closely with the Yukthi module whenever domain-specific knowledge is required. Before generating a response, Yukthi determines whether external reference material is necessary and retrieves relevant information from an agricultural knowledge base. Retrieved content is incorporated into the language model's context through a Retrieval-Augmented Generation (RAG) framework, reducing the likelihood of hallucinated recommendations and improving the trustworthiness of generated advice.

The Shared Foundation Layer acts as the underlying infrastructure supporting all modules. It manages user authentication, database operations, environmental configuration, geospatial services, weather retrieval, and data persistence. By centralising these cross-cutting concerns, the architecture avoids duplication and promotes consistency across the platform.

The operational workflow of NAVA begins with user interactions through the frontend interface. Depending on the requested operation, the system routes incoming requests to disease detection services, VNIR monitoring services, farm management services, or conversational AI services. Information generated by these services is continuously persisted within the platform's storage layer, enabling long-term tracking of farm activities and crop health conditions.

A key architectural characteristic of NAVA is its emphasis on contextual continuity. Unlike conventional agricultural applications that process each request independently, NAVA maintains a persistent understanding of farm conditions across multiple interactions. Disease diagnoses,

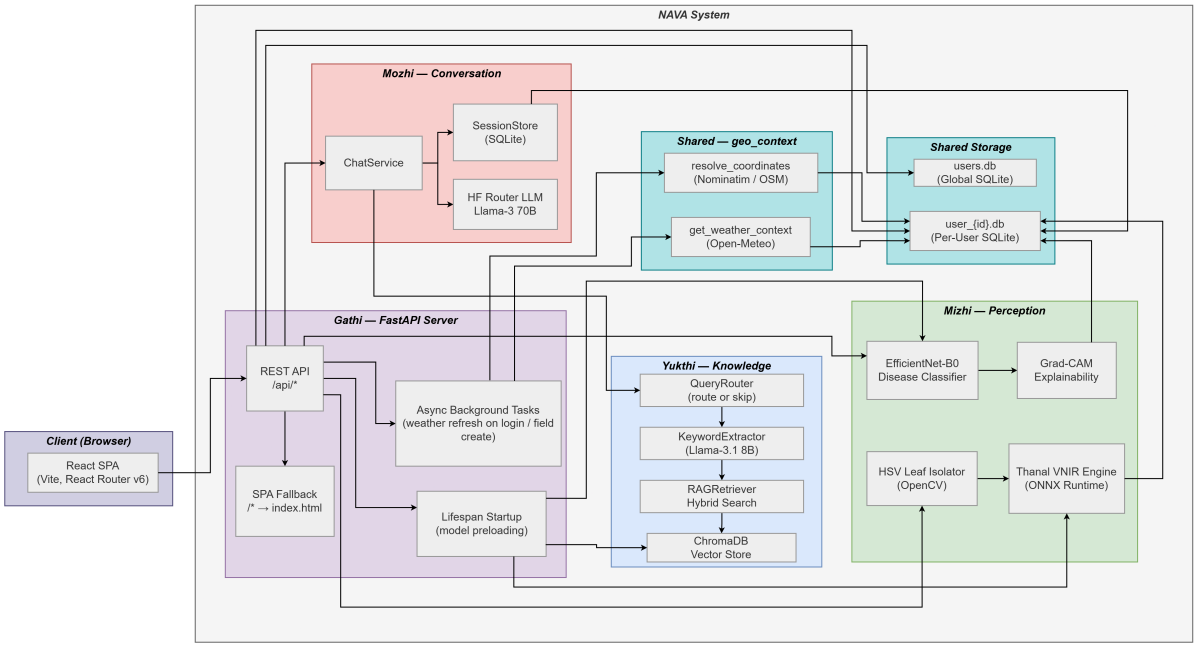

Figure 1: High-Level Architecture of NAVA

monitoring results, environmental observations, user notes, and previous conversations collectively contribute to an evolving farm profile that informs future recommendations. This approach enables the platform to transition from a reactive diagnostic tool into a proactive agricultural assistant capable of supporting long-term decision making.

The architecture also prioritises transparency and reliability. Explainable AI mechanisms provide visual justification for disease predictions, while retrieval-augmented generation ensures that advisory responses are grounded in verified agricultural references. Together, these design principles contribute to the development of a trustworthy digital agronomist that can assist farmers throughout the crop lifecycle.

3.2 Data Collection and Preprocessing

The performance, reliability, and safety of the NAVA platform depend heavily on the quality and diversity of the data used to build its core models. Since NAVA combines computer vision, virtual spectral analysis, and retrieval-augmented conversational intelligence, each subsystem required a dedicated data collection and preprocessing strategy. The overall aim was to ensure that the models could operate robustly under real-world agricultural conditions, where lighting, background clutter, image quality, and crop variability are often unpredictable.

To achieve this, a multi-source data collection approach was adopted. Separate datasets were curated for disease detection, VNIR prediction, and the RAG knowledge base. Each dataset was then processed using pipelines tailored to the specific requirements of the corresponding subsystem.

3.2.1 Disease Detection

The disease detection module is responsible for identifying crop diseases from leaf images captured under field conditions. Because real-world farm imagery is much more variable than controlled benchmark data, a broad and balanced training corpus was required to improve generalisation.

Data Collection Superset Strategy: Instead of relying on a single dataset, a Superset was constructed by combining multiple publicly available agricultural repositories. This aggregation strategy increased the diversity of disease appearances, crop species, imaging conditions, and acquisition devices. The Superset covers 34 disease classes across 7 major crops, together with healthy baseline classes for the same crops.

Table 1: Sources Used in the Disease Detection Superset

Dataset Source	**Role in the Superset**
PlantVillage [22]	Controlled laboratory crop disease images
PlantWild V1 and V2 [23]	Field-condition imagery with real-world variation
PlantDoc [24]	Diverse field images with cluttered backgrounds
PaddyDoctor [25]	Rice-specific disease images relevant to Kerala agriculture
ASDID [26]	Soybean disease imagery
Kaggle agricultural datasets	Additional crop and disease samples

The resulting dataset provides broad coverage of practical agricultural scenarios. By combining laboratory-quality samples with field-acquired images, the model learns features that are more resilient to changes in environment and camera quality.

Data Preprocessing and Balancing: The initial aggregated dataset exhibited severe class imbalance. Some classes contained more than 5,700 images, while others had fewer than 71. Such imbalance can bias the model toward majority classes and reduce predictive performance on underrepresented diseases.

To address this, a strict *300–700 Filtering Rule* was applied. Classes with fewer than 300 images were removed, while classes with more than 700 images were downsampled. This filtering stage removed extremely weak classes and prevented dominant classes from overwhelming the training process. As a result, the dataset became significantly more balanced and better suited for supervised learning.

After filtering, the minority classes were expanded using the Albumentations library. The augmentation pipeline was designed to simulate realistic field conditions and improve robustness. The transformations included horizontal flips, random rotations, affine scaling in the range of 0.9–1.1, brightness and contrast adjustments, RGB shifts, and Gaussian blur. These operations help the model become less sensitive to lighting changes, focus issues, and sensor variation.

To support downstream processing, all class names were standardised using a snake_case naming

Table 2: Final Disease Detection Dataset Statistics

Parameter	**Value**
Number of crops	7
Number of disease classes	34
Training samples	17,340
Validation samples	3,060
Testing samples	4,089
Total training and validation samples	20,400

convention in the format `crop_disease`. For example, the corn disease class Northern Leaf Blight was stored as `corn_northern_leaf_blight`. This naming pattern simplifies metadata handling and supports automatic token extraction for the language-based components of NAVA.

At inference time, images are processed using a standard preprocessing pipeline. Each image is resized so that the shorter edge becomes 256 pixels, centre-cropped to 224×224, converted to a tensor, and normalised using standard ImageNet statistics. This ensures that training and inference follow a consistent data format.

3.2.2 NIR Prediction

A major feature of NAVA is its ability to estimate virtual near-infrared (VNIR) information from ordinary RGB images. This makes it possible to monitor physiological plant stress without requiring specialised spectral sensors.

Data Collection: To train the VNIR estimation model, a paired RGB-NIR dataset was required. Such datasets are relatively rare because aligned spectral capture demands additional hardware and controlled acquisition. For this project, the dataset from *deepNIR: Datasets for Generating Synthetic NIR Images and Improved Fruit Detection System Using Deep Learning Techniques* [27] was used. The Capsicum RGB-NIR pair dataset was selected because it offers high-quality alignment between RGB images and corresponding NIR targets.

Table 3: Capsicum RGB-NIR Dataset Characteristics

Parameter	**Value**
Dataset	deepNIR Capsicum RGB-NIR pair dataset
Total image pairs	1,615
Image resolution	1280×960 pixels
Image precision	8-bit
Training split	80%
Testing split	10%
Validation split	10%

Data Preprocessing: The VNIR model was trained directly on the raw full-frame RGB images and their corresponding NIR targets. The preprocessing steps were intentionally kept simple and spatially consistent. First, both the RGB inputs and NIR targets were resized to 256×256 pixels. Next, pixel values were scaled from the range [0,255] to [0,1] and converted to 32-bit floating-point representation. Finally, the image tensors were converted from HWC format to CHW format, which is required by the network architecture.

This preprocessing pipeline preserved the spatial and spectral correspondence between the input and output images while ensuring that the data format was compatible with the training process.

3.2.3 RAG Database

The conversational component of NAVA relies on Retrieval-Augmented Generation (RAG) to ensure that agricultural advice is grounded in verified reference material. This prevents the language model from generating unsupported recommendations such as incorrect chemical dosages or unsuitable crop practices.

Data Collection: The knowledge base was compiled from authoritative agricultural extension documents and educational resources. The documents were organised by crop so that retrieval would remain domain-specific and avoid cross-crop contamination.

Table 4: Crop-Wise Sources Used in the RAG Knowledge Base

Crop	**Primary Sources**
Banana	PlantVillage
Cassava	PlantVillage
Corn	PlantVillage
Cucumber	Cornell University, Nova Scotia Vegetable, Perennia Food and Agriculture Corporation, PlantVillage
Rice	Louisiana State University, PlantVillage, Tamil Nadu Agricultural University
Soybean	Crop Protection Network, PlantVillage, Tamil Nadu Agricultural University
Tomato	Cornell University, Missouri Botanical Garden, PlantVillage, Tamil Nadu Agricultural University, University of Minnesota, University of Wisconsin–Madison

Data Preprocessing and Ingestion Pipeline The raw `.txt` and `.pdf` documents were converted into a searchable vector database using an automated ingestion pipeline. The process consisted of three stages.

First, the documents were chunked. Text files were split into paragraphs using double-newline boundaries, and overly long paragraphs were further segmented using a token limit. The chunker also detected section headers, such as lines ending in colons or using markdown-style heading

formatting, and stored them as metadata to preserve document structure. PDF files were parsed using PyMuPDF (fitz), which extracted page-level and block-level text before applying the same chunking logic.

Second, each text chunk was converted into a dense vector representation using the BAAI/bge-small-en-v1.5 sentence transformer model. This embedding model produces 384-dimensional vectors and was applied in batches of 64 for efficient processing.

Third, the generated embeddings, raw text, and metadata were stored persistently in crop-specific ChromaDB collections such as `nava_banana` and `nava_rice`. Metadata fields included the crop name, source filename, section title, and chunk index. Deterministic identifiers based on the source filename and chunk index were used during upsertion, making the ingestion process idempotent and safe to rerun without duplicating entries.

Through this pipeline, static documents were transformed into a structured and searchable knowledge base for the RAG subsystem. The resulting corpus enables NAVA to retrieve relevant passages during conversation and generate responses that remain grounded in authoritative agricultural references.

3.3 Gathi: API Server and Frontend Integration

Gathi serves as the orchestration layer of the NAVA platform and acts as the primary interface between end users and the underlying intelligent services. All interactions with the system, including disease diagnosis, VNIR-based stress monitoring, conversational assistance, farm management operations, authentication, and weather-related services, are routed through Gathi. The module was designed to provide a unified access layer that abstracts the complexity of the underlying subsystems while maintaining modular separation between perception, reasoning, knowledge retrieval, and data management components. The architecture of Gathi consists of two tightly integrated components: a FastAPI-based backend that exposes the system functionality through RESTful services and a React-based single-page application (SPA) that provides the user interface. Together, these components establish a cohesive full-stack framework for delivering agricultural intelligence services.

3.3.1 FastAPI Backend Framework

The backend component of Gathi was implemented using FastAPI, a modern Python web framework designed for high-performance API development. FastAPI was selected due to its asynchronous request handling capabilities, automatic request validation through Pydantic schemas, dependency injection framework, and support for scalable service-oriented architectures [28]. Within NAVA, the backend functions as both an API gateway and a coordination layer that manages communication between the frontend and the specialized modules of the platform.

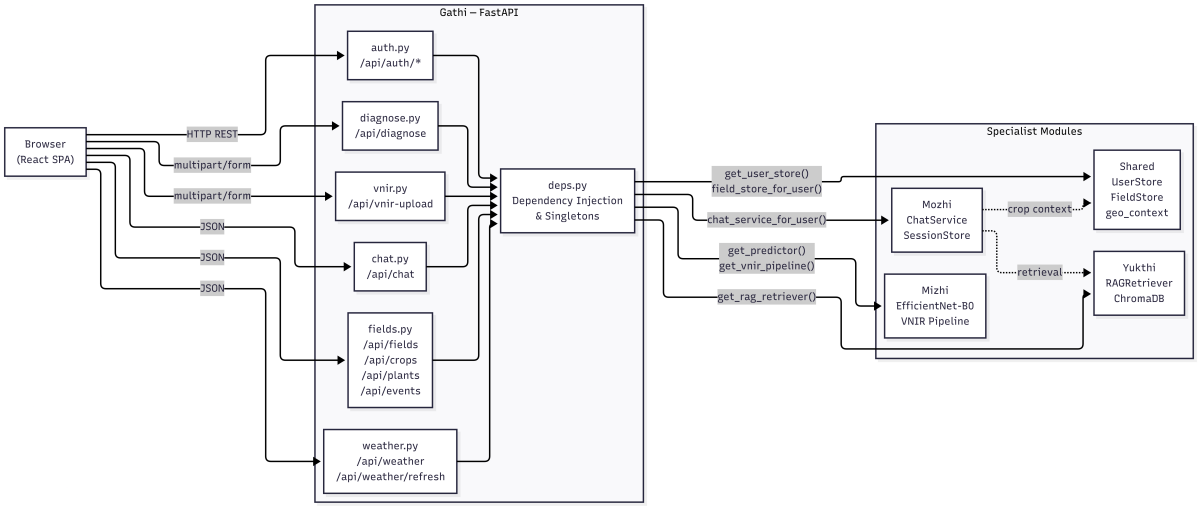

Figure 2: High-Level Architecture of Gathi

The FastAPI application is initialized within the `main.py` module, where middleware, routing logic, startup procedures, and frontend serving mechanisms are configured. One of the primary responsibilities of the backend is the registration of API routers corresponding to different system functionalities. Authentication services, disease diagnosis endpoints, VNIR monitoring services, conversational AI interactions, farm management operations, and weather services are each encapsulated within dedicated routers. This modular routing strategy promotes separation of concerns and simplifies maintenance and future extension of the platform.

To support frontend-backend communication during development, Cross-Origin Resource Sharing (CORS) middleware is configured to permit requests from local frontend development servers. Although production deployment serves both frontend and backend from a common origin, the middleware ensures compatibility during development and testing phases. The backend therefore maintains a consistent communication interface regardless of deployment configuration.

A significant architectural consideration within Gathi is the management of computationally intensive resources during system startup. NAVA relies on several large components, including the EfficientNet-B0 disease classification model, the VNIR estimation pipeline, and the ChromaDB-based retrieval infrastructure. Loading all resources synchronously would substantially increase system startup latency and delay service availability. To address this issue, Gathi employs FastAPI's lifespan management mechanism combined with a hybrid initialization strategy. The ChromaDB vector database is initialized within the main execution thread due to underlying native library constraints, while the EfficientNet-B0 and VNIR models are loaded asynchronously in background daemon threads. This design enables the API server to begin accepting requests while model loading continues in parallel, thereby reducing perceived startup time and improving deployment responsiveness.

To facilitate efficient resource utilization, Gathi employs FastAPI's dependency injection mech-

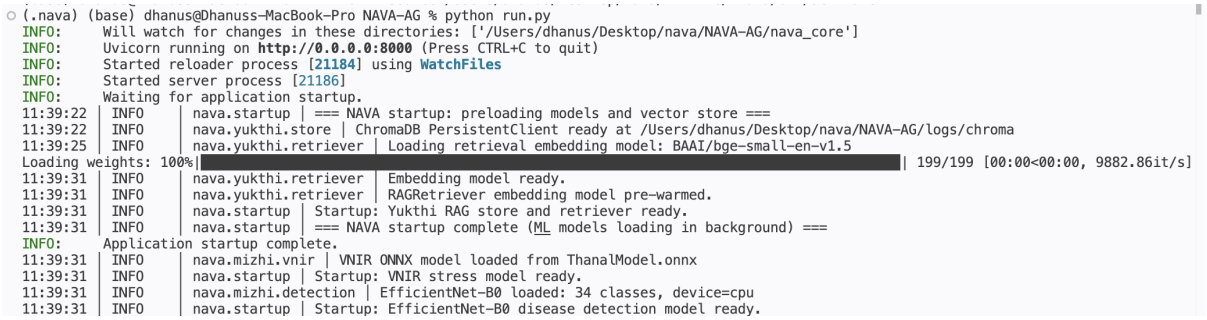

Figure 3: Gathi Startup Sequence and Model Initialization Workflow

anism. Frequently used services such as the disease prediction engine, VNIR pipeline, configuration manager, user database interface, and retrieval services are instantiated as process-wide singleton objects using memoized dependency functions. This approach avoids repeated initialization of expensive resources and ensures consistent object access throughout the application lifecycle. Request-specific services, including user authentication, field-specific database access, session management, and chat service construction, are generated dynamically through dependency resolution. Such a design enhances modularity and enables clean integration between independently developed subsystems.

Authentication and authorization are implemented through token-based access control. User credentials are stored securely using bcrypt hashing, while session tokens are generated during registration and login operations. Protected endpoints automatically validate incoming tokens through dedicated dependency functions before processing requests. This mechanism ensures that access to farm records, crop information, monitoring histories, and conversational sessions remains restricted to authenticated users. Data isolation is further strengthened through user-specific database structures, preventing unauthorized access to agricultural records belonging to other users.

The backend exposes a comprehensive collection of REST endpoints that collectively support all major system functionalities. Authentication routers manage user registration, login, profile updates, and account maintenance. Diagnostic endpoints coordinate image upload, disease classification, and Grad-CAM generation. VNIR endpoints support stress monitoring workflows and historical record management. Conversational endpoints facilitate interaction with the Mozhi module, while field and weather endpoints support farm management operations and contextual environmental data retrieval. By centralizing these services under a unified API layer, Gathi provides a consistent and extensible integration framework for the entire NAVA platform.

From an architectural perspective, Gathi deliberately avoids embedding domain-specific agricultural intelligence within the API layer. Instead, it functions as a coordination and orchestration framework that delegates specialized operations to Mizhi, Mozhi, Yukthi, and the Shared Foundation Layer. This separation preserves modularity, improves maintainability, and allows

Table 5: Gathi API Routers and Supported Endpoints

Router	**Primary Functionality**	**Key Endpoints**
Auth Router	User authentication and account management	`/api/auth/register`, `/api/auth/login`, `/api/auth/logout`, `/api/auth/me`, `/api/auth/password`
Diagnose Router	Disease diagnosis using EfficientNet with Grad-CAM visualization	`/api/diagnose`
VNIR Router	Leaf stress analysis using VNIR pipeline and history management	`/api/vnir-upload`, `/api/vnir-clear`
Chat Router	Conversational assistant with RAG context, history, and memory summaries	`/api/chat`, `/api/chat/history`, `/api/chat/clear`, `/api/chat/summary`
Fields Router	Field, crop, plant, event, and contextual farm data management	`/api/fields`, `/api/crops`, `/api/plants`, `/api/field-context`, `/api/crop-context`, `/api/events`, `/api/field-notes`
Weather Router	Weather retrieval and refresh using Open-Meteo API	`/api/weather`, `/api/weather/refresh`

individual subsystems to evolve independently without requiring extensive modifications to the communication infrastructure.

3.3.2 React Single Page Application (SPA)

The frontend component of Gathi was developed as a React-based Single Page Application (SPA) using React 18 and the Vite build system. The frontend serves as the primary interaction medium through which farmers access the capabilities of NAVA. The SPA architecture was selected to provide responsive user experiences, seamless client-side navigation, and efficient communication with backend services through asynchronous API requests [29].

The application is organized around a component-driven architecture that separates reusable interface elements, application pages, utility functions, and state management mechanisms. Routing is implemented using React Router, enabling navigation between authentication screens, farm dashboards, crop workspaces, field management interfaces, and user profile pages without

requiring full-page reloads. This design improves user experience while reducing server-side rendering overhead.

At application startup, the React root is initialized and wrapped within a browser routing context. Route definitions specify both public and protected application pages. Public routes include the landing page and authentication interface, while authenticated routes provide access to field management, crop-specific workspaces, and profile management features. Access control is enforced through a route-guard mechanism that validates user authentication status before rendering protected content. Unauthenticated users attempting to access secured resources are redirected to the authentication interface.

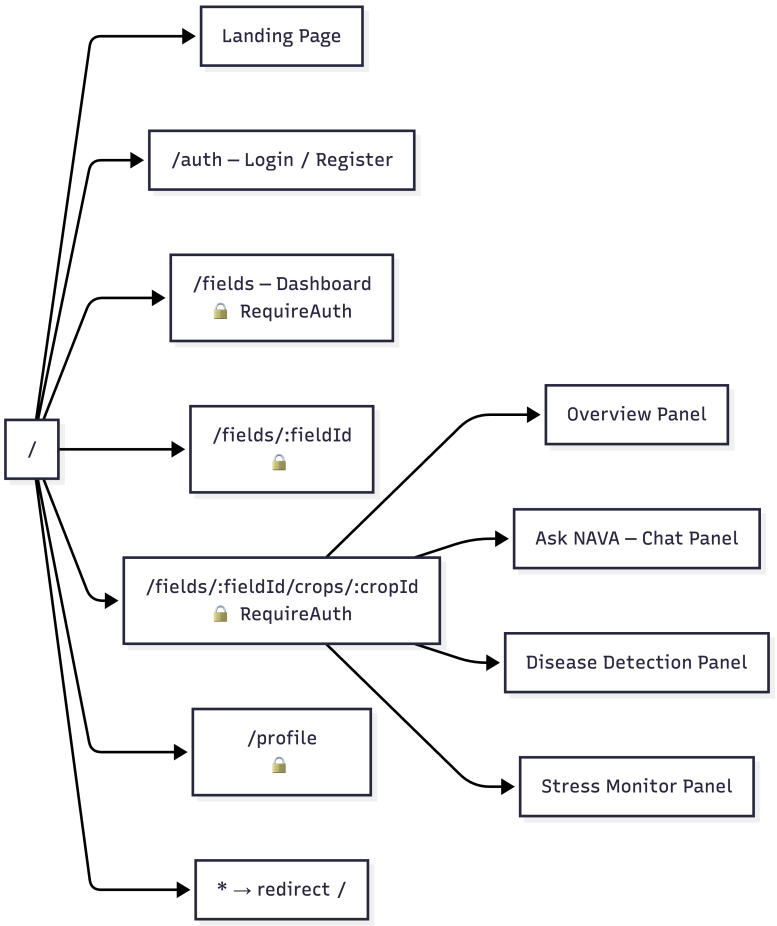

Figure 4: Frontend Route Hierarchy of NAVA React Application

Authentication state management is implemented using a dedicated React context provider. The context maintains information regarding the currently authenticated user, loading status, and session control functions. Authentication tokens and user metadata are persisted within browser local storage, enabling session continuity across browser refreshes. During application initialization, stored credentials are validated through backend API calls to ensure that expired or invalid sessions are removed automatically. This mechanism provides a balance between usability and security while minimizing unnecessary login operations.

Communication between the frontend and backend is centralized through a custom API abstraction layer. Rather than issuing direct HTTP requests throughout the application, all service interactions pass through a shared request wrapper that automatically attaches authentication tokens, handles error responses, and standardizes response processing. This abstraction reduces code duplication and provides a consistent communication mechanism across all application modules.

The visual structure of the SPA is organized through reusable layout components. Shared navigation elements, authentication controls, and page containers are encapsulated within common layout structures that ensure interface consistency across the application. Specialized layouts are used within crop-level workspaces to accommodate the more complex interaction requirements of disease diagnosis, VNIR monitoring, conversational assistance, and crop management operations. This component-based approach improves maintainability while supporting future interface extensions.

The frontend is designed to function as a thin client that delegates all computationally intensive operations to backend services. Disease diagnosis, VNIR inference, retrieval-augmented generation, weather retrieval, and memory-aware conversational processing are all executed server-side. The SPA is therefore responsible primarily for user interaction, data visualization, request submission, and result presentation. Such a separation ensures that computational requirements remain independent of client hardware capabilities and allows users to access advanced agricultural intelligence services through standard web browsers.

In production deployment, the compiled React application is served directly by the FastAPI backend. Static assets are delivered through dedicated asset routes, while all non-API requests are redirected to the SPA entry point. This deployment model simplifies system administration by allowing both frontend and backend components to operate under a single server process and common origin. Consequently, the architecture reduces infrastructure complexity while preserving clear separation between presentation and application logic.

Overall, the React SPA provides an accessible and responsive interface through which users interact with the NAVA ecosystem, while the FastAPI backend supplies the orchestration and integration mechanisms required to connect the perception, reasoning, retrieval, and farm management subsystems into a unified agricultural intelligence platform.

3.4 Mizhi: Vision Pipeline and Proactive Monitoring

Mizhi constitutes the perception layer of the NAVA platform and is responsible for transforming raw visual inputs into actionable agricultural intelligence. The module provides two complementary capabilities: disease identification through deep learning–based image classification and proactive physiological stress monitoring through virtual near-infrared (VNIR) estimation. Together, these capabilities enable both reactive diagnosis of visible crop diseases and early detection of stress conditions before observable symptoms emerge.

The design of Mizhi was motivated by the limitations of conventional agricultural monitoring approaches. Traditional disease diagnosis relies on visible symptoms, which often appear only after significant physiological damage has occurred. Similarly, manual field inspection is constrained by labor requirements, human subjectivity, and limited access to expert agronomists. To address these challenges, Mizhi integrates computer vision techniques, explainable artificial intelligence, and spectral estimation methods within a unified perception framework.

The module is organized into two primary subsystems. The first subsystem performs disease classification using an EfficientNet-B0 convolutional neural network supported by Grad-CAM explainability. The second subsystem, referred to as Thanal, estimates near-infrared reflectance from RGB using an Attention U-Net model and analyzes temporal trends to identify physiological stress. Both pipelines share a common image acquisition workflow and store their outputs within the farm event repository, enabling subsequent utilization by other NAVA modules such as Mozhi and Yukthi.

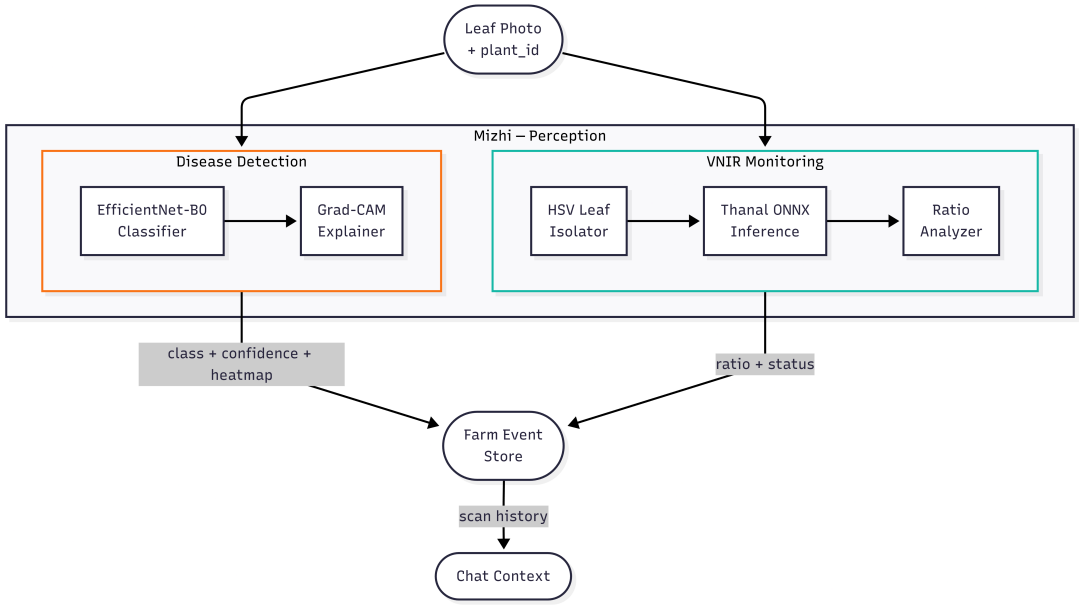

Figure 5: High-Level Architecture of Mizhi

3.4.1 Disease Classification with EfficientNet-B0

The disease classification component of Mizhi was developed to identify crop diseases directly from leaf images captured using standard smartphone cameras. The objective was to create a robust classification system capable of operating under real-world agricultural conditions while maintaining computational efficiency suitable for deployment within the NAVA platform.

To improve generalization performance, the model was trained using a consolidated dataset constructed from multiple publicly available agricultural image repositories. Rather than relying on a single benchmark dataset, a superset strategy was adopted in which samples from PlantVillage, PlantWild, PlantDoc, PaddyDoctor, ASDID, and additional agricultural datasets

were combined into a unified corpus. This approach increased visual diversity and exposed the model to a broader range of environmental conditions, lighting variations, disease manifestations, and crop species. The resulting dataset covered thirty-four disease categories distributed across seven major crops, including rice, corn, tomato, soybean, cassava, banana, and cucumber. Healthy plant classes were also incorporated to support differentiation between diseased and non-diseased specimens.

To address class imbalance, a balancing strategy was employed during dataset preparation. Classes containing insufficient samples were excluded to ensure adequate representation during learning, while highly populated classes were downsampled to prevent bias toward majority categories. Data augmentation techniques were subsequently applied to increase robustness against environmental variability. Transformations included geometric modifications, brightness and contrast adjustments, color perturbations, and image blurring effects. These augmentations simulated common field conditions such as inconsistent illumination, camera movement, and device-specific color variations.

EfficientNet-B0 is a convolutional neural network architecture proposed by Tan and Le (2019) that introduces a compound scaling methodology for improving model performance while maintaining computational efficiency. Unlike conventional approaches that independently increase network depth, width, or input resolution, EfficientNet scales all three dimensions in a balanced manner using a fixed set of scaling coefficients. The EfficientNet-B0 model serves as the baseline architecture of the EfficientNet family and is built using mobile inverted bottleneck convolution (MBConv) blocks combined with squeeze-and-excitation (SE) modules, enabling effective feature extraction with a relatively small number of parameters. This design allows the network to achieve strong classification performance while remaining computationally efficient, making it suitable for practical deployment scenarios [30].

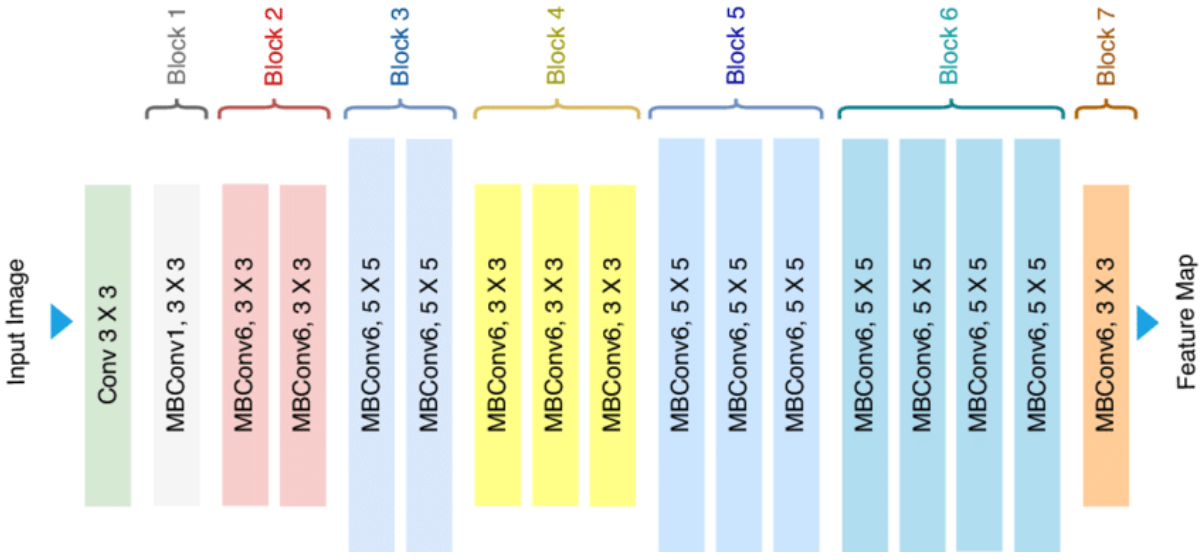

Figure 6: Architecture of EfficientNet-B0

The selection of EfficientNet-B0 was based on a comparative evaluation involving multiple convolutional neural network architectures. Experimental analysis demonstrated that EfficientNet-B0 achieved superior classification performance while maintaining relatively low computational complexity. Unlike conventional scaling strategies that focus exclusively on network depth or width, EfficientNet employs compound scaling to jointly optimize network depth, width, and image resolution. This balanced scaling methodology enables improved feature extraction efficiency and stronger predictive performance without substantial increases in computational cost. Experimental evaluation reported a classification accuracy of 94.54%, outperforming alternative architectures evaluated under identical conditions.

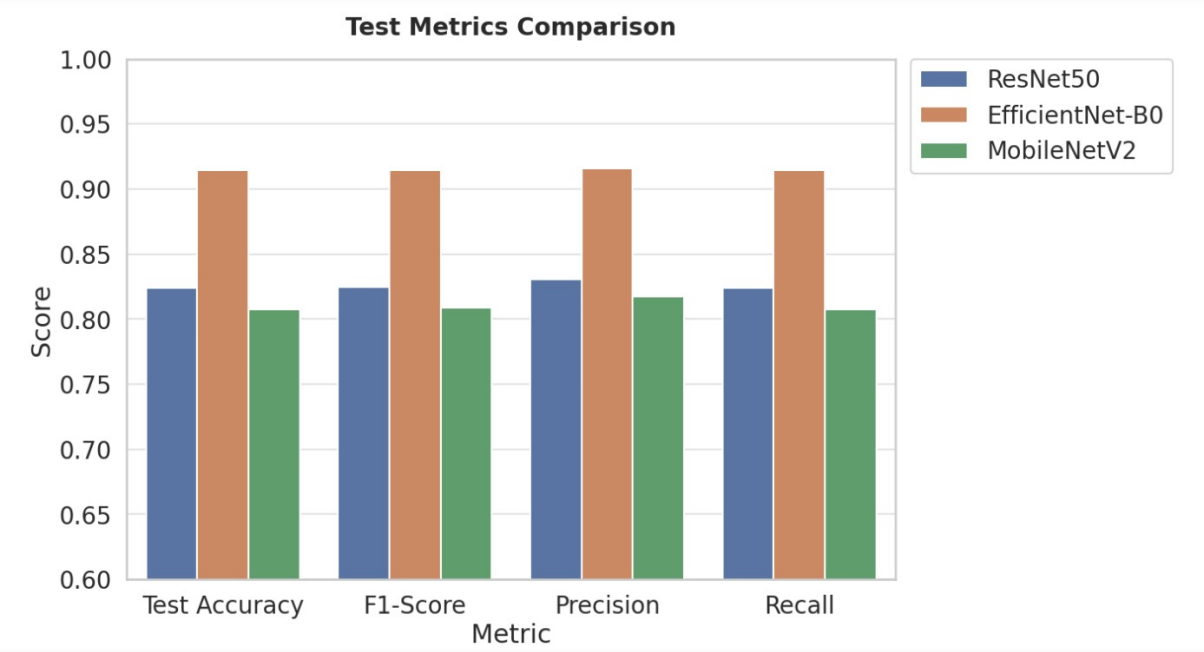

Figure 7: Comparative Performance of Evaluated Classification Models

Within the deployed system, disease classification is implemented through the EfficientNetB0 Predictor component. During initialization, the model architecture is reconstructed, trained weights are loaded from a checkpoint file, and the classifier head is configured to match the number of disease categories supported by the platform. To maximize compatibility, the implementation supports multiple checkpoint formats, allowing the model to be restored regardless of the serialization strategy employed during training. Once loaded, the network is transferred to the configured computation device and switched to evaluation mode to disable training-specific operations.

Before inference, input images undergo a standardized preprocessing pipeline. Images are resized, center-cropped to the expected input dimensions, converted into tensors, and normalized using ImageNet statistics. This preprocessing procedure ensures consistency between training and deployment conditions while reducing sensitivity to image acquisition differences.

The inference workflow generates probability scores for all disease categories using a softmax

activation function. The class associated with the highest probability is selected as the predicted diagnosis. To improve reliability and mitigate risks associated with incorrect recommendations, Mizhi incorporates a confidence-based safety mechanism. Predictions whose confidence values fall below a predefined threshold are labeled as unreliable. Such predictions remain accessible to users but are explicitly marked to indicate reduced certainty. This design prevents low-confidence classifications from being interpreted as definitive diagnoses and encourages additional verification when necessary.

3.4.2 Explainability Layer using Grad-CAM

Although deep convolutional neural networks provide strong predictive performance, their internal decision-making processes are often difficult to interpret. In agricultural applications, explainability is particularly important because farmers and agricultural practitioners must understand the rationale behind automated recommendations before acting upon them. To address this requirement, Mizhi incorporates Gradient-weighted Class Activation Mapping (Grad-CAM) as an explainability layer.

Grad-CAM generates visual explanations by highlighting image regions that contributed most strongly to the predicted class. Instead of presenting only a disease label and confidence score, the system also provides a spatial heatmap indicating the areas of the leaf that influenced the classification process. This additional information improves transparency and increases user confidence in the model's predictions [31].

The implementation utilizes a hook-based mechanism attached to the final convolutional feature extraction layer of EfficientNet-B0. During the forward pass, activation maps from the selected layer are recorded. During backpropagation, gradients corresponding to the predicted class are captured. These gradients are subsequently aggregated to determine the relative importance of each feature channel. The weighted activation maps are combined and processed using a rectified linear activation function to retain only positive contributions to the prediction.

The resulting activation map is normalized and resized to match the dimensions of the original image. A color-coded heatmap is then generated and superimposed on the input image to produce an interpretable visual representation of model attention. Regions receiving higher attention scores are displayed with warmer colors, indicating stronger influence on the final classification decision. The generated visualization is returned alongside the prediction result and stored as part of the diagnostic event record.

An important design consideration is that Grad-CAM generation is performed only for reliable predictions. Low-confidence classifications bypass the explainability stage because visual explanations derived from uncertain predictions may be misleading. This selective execution strategy reduces computational overhead while ensuring that generated explanations correspond to predictions with acceptable confidence levels.

The integration of Grad-CAM transforms the disease classification system from a purely predictive model into an interpretable decision-support tool. By revealing the image regions that influenced model behavior, the explainability layer facilitates validation of predictions, supports trust development among users, and provides an additional safeguard against inappropriate diagnostic recommendations.

3.4.3 VNIR Estimation Engine (Thanal)

While disease classification focuses on visible symptoms, physiological stress often develops before any observable signs appear on the plant surface. Consequently, a complementary monitoring mechanism is required to identify early-stage stress conditions. The Thanal subsystem was developed to address this requirement by estimating near-infrared reflectance from conventional RGB imagery.

Near-infrared reflectance is widely recognized as an indicator of plant health because healthy vegetation typically exhibits strong reflectance in the NIR spectrum due to internal cellular structures. Stress conditions, nutrient deficiencies, disease progression, and environmental disturbances can alter these reflectance characteristics before visible symptoms become apparent. Traditional acquisition of NIR information requires specialized multispectral imaging equipment, which is often prohibitively expensive for smallholder farmers. Thanal addresses this limitation by generating a virtual NIR representation using a deep learning model operating on standard RGB images.

U-Net is a convolutional neural network architecture originally proposed by Ronneberger et al. (2015) for biomedical image segmentation and has since been widely adopted for image-to-image translation and reconstruction tasks. The architecture follows an encoder-decoder structure in which the encoder progressively extracts high-level feature representations through a series of convolution and downsampling operations, while the decoder reconstructs the target image through upsampling and feature refinement stages. A key characteristic of U-Net is the use of skip connections that directly transfer feature maps from encoder layers to their corresponding decoder layers, allowing fine-grained spatial information to be preserved during reconstruction. This combination of contextual feature learning and spatial detail retention makes U-Net particularly effective for dense prediction tasks [32].

The core of the Thanal system is a UNet architecture enhanced with attention gate mechanisms. The encoder-decoder structure of UNet enables effective extraction of both global contextual information and local spatial details, while attention gates improve focus on relevant leaf regions during reconstruction. The model accepts an RGB image as input and produces a grayscale output representing estimated near-infrared reflectance values at the pixel level. Validation experiments reported a peak signal-to-noise ratio (PSNR) of approximately 28 dB and a structural similarity index (SSIM) of approximately 0.85, indicating strong agreement between estimated and reference NIR representations.

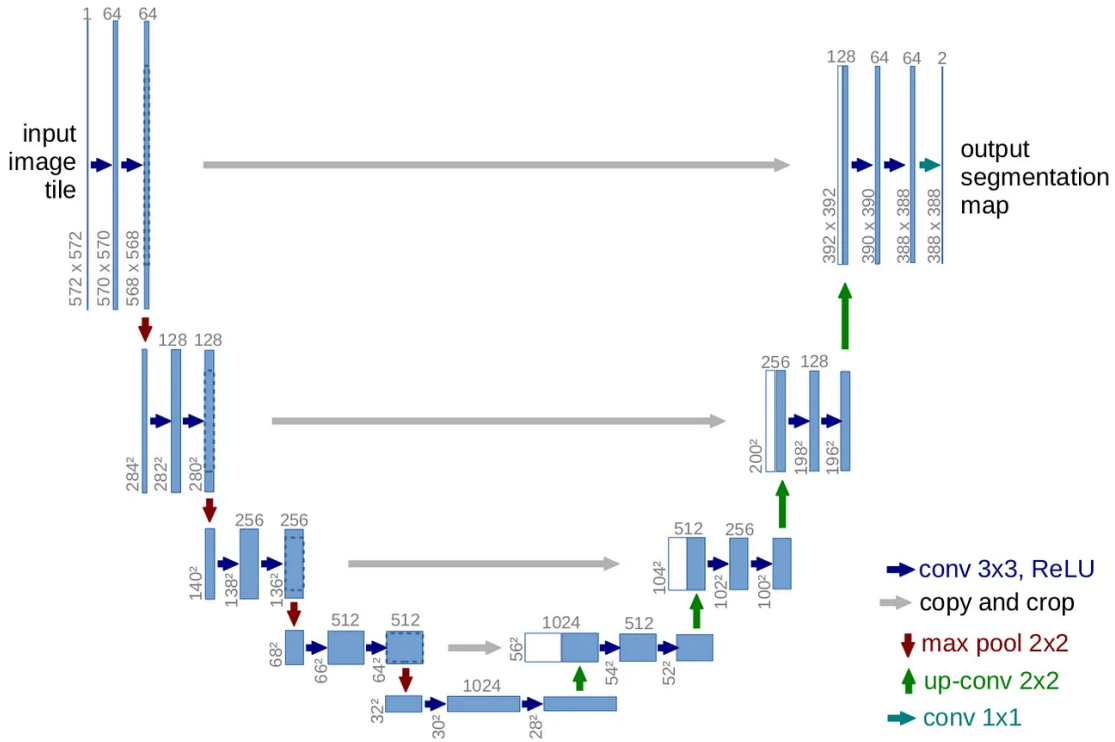

Figure 8: Architecture of U-Net

For deployment efficiency, the trained model was exported to the Open Neural Network Exchange (ONNX) format and executed using ONNX Runtime. This deployment strategy enables lightweight CPU-based inference and reduces dependency on GPU hardware. The architecture was additionally validated on resource-constrained edge computing devices, demonstrating the feasibility of practical agricultural deployment scenarios.

Prior to VNIR estimation, images undergo a leaf isolation stage designed to remove background information and retain only relevant plant tissue. The procedure operates in the HSV color space, where vegetation can be more effectively separated from surrounding objects. Multiple color masks are applied to identify healthy green regions and yellow-brown tissue associated with advanced stress. Morphological operations and contour analysis are subsequently employed to refine segmentation results and isolate the dominant leaf region.

The leaf isolation stage categorizes images into three possible states. Images containing healthy green leaf tissue proceed to VNIR estimation. Images dominated by yellow-brown vegetation are immediately classified as exhibiting severe visual stress, eliminating the need for further spectral estimation. Images in which no leaf structure is detected are flagged accordingly and excluded from subsequent analytical processing. This preliminary screening stage improves computational efficiency and enhances robustness against invalid inputs.

Following segmentation, the isolated leaf image is passed to the ONNX inference engine, which generates a grayscale VNIR map. Bright regions within the generated image correspond

to stronger estimated near-infrared reflectance and therefore indicate healthier plant tissue. Darker regions suggest reduced reflectance and potentially elevated stress levels. The generated VNIR representation serves as the basis for quantitative stress analysis within the subsequent monitoring stage.

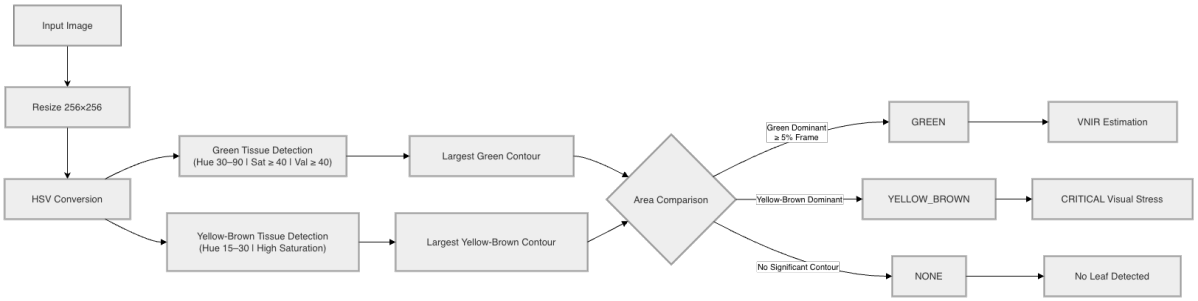

Figure 9: HSV Isolation Process

The VNIR analyzer computes average green-channel intensity from the RGB image and average estimated NIR intensity from the generated VNIR map. These measurements are combined into a VNIR-to-green ratio, which serves as a normalized indicator of plant physiological status. Rather than relying on fixed species-dependent thresholds, the system evaluates this ratio relative to the historical measurements of the same plant, thereby supporting adaptive monitoring across different crop types and environmental conditions.

3.4.4 Rolling Baseline Stress Detection Logic

A central challenge in stress monitoring is the absence of universally applicable threshold values. Reflectance characteristics vary significantly across plant species, growth stages, environmental conditions, and imaging devices. Consequently, using a fixed threshold can lead to unreliable assessments and poor generalization across agricultural contexts.

To overcome this limitation, Mizhi employs a relative historical comparison strategy known as rolling baseline stress detection. Rather than comparing current measurements against predefined reference values, the system evaluates each plant against its own historical behavior. This approach enables personalized monitoring and eliminates the need for crop-specific calibration procedures.

The monitoring framework begins by constructing a baseline from the first five valid VNIR ratio measurements collected for a plant. These initial observations are assumed to represent healthy operating conditions and are therefore used to establish a reference state. Measurements in which no leaf is detected are excluded from baseline construction to prevent contamination of statistical estimates.

As new observations are acquired, the system maintains a rolling window containing the most recent valid measurements. Statistical summaries including baseline averages, rolling averages,

checkpoint averages, and ratio change percentages are continuously updated. The monitoring algorithm then evaluates two independent conditions corresponding to different levels of stress severity.

The first condition is a warning-level assessment. The current VNIR ratio is compared against the rolling average of recent observations. If the reduction exceeds a predefined threshold, the system generates a warning indicating a potentially deteriorating physiological trend. This mechanism provides early notification of emerging stress before substantial degradation occurs.

The second condition is a critical-level assessment. The current ratio is compared against the original baseline established during the healthy reference period. A sufficiently large reduction relative to this baseline triggers a critical alert indicating a significant departure from normal physiological conditions. Because this comparison references the plant's historical healthy state, it serves as a stronger indicator of substantial stress development.

When both warning and critical conditions are satisfied simultaneously, the critical alert takes precedence. This prioritization ensures that severe stress conditions are communicated clearly and without ambiguity. Additionally, images classified as containing visibly stressed yellow-brown tissue are immediately assigned a critical visual stress status irrespective of ratio-based calculations, reflecting the direct evidence of physiological deterioration observed during leaf segmentation.

The monitoring framework supports multiple operational states, including calibration mode, normal operation, warning-level stress detection, critical stress detection, visual stress identification, and no-leaf detection. Each status is stored within the plant history database together with associated statistical measurements, enabling long-term temporal analysis and integration with the conversational and decision-support capabilities of the broader NAVA ecosystem.

By combining VNIR estimation with historical trend analysis, the rolling baseline framework enables Mizhi to function not only as a diagnostic tool but also as a proactive monitoring system capable of identifying physiological changes before visible disease symptoms emerge. This capability extends the role of NAVA beyond reactive disease identification and supports early intervention strategies aimed at reducing crop losses and improving farm management outcomes.

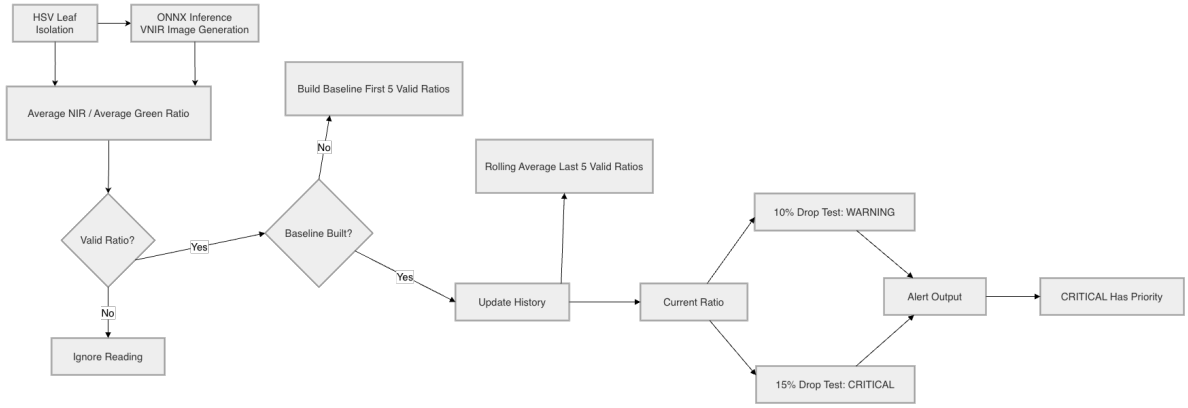

Figure 10: VNIR Estimation and Stress Monitoring Pipeline

3.5 Mozhi: Conversational AI and Memory Management

Mozhi functions as the cognitive layer of the NAVA platform and is responsible for enabling natural language interaction between users and the system. While Mizhi focuses on perception and Yukthi provides knowledge retrieval capabilities, Mozhi serves as the decision orchestration component that integrates conversational intelligence, contextual reasoning, memory management, and knowledge grounding into a unified dialogue framework.

The primary objective of Mozhi is to transform NAVA from a collection of independent analytical tools into a persistent digital agronomist capable of maintaining meaningful long-term interactions with farmers. Rather than responding solely to isolated user queries, Mozhi continuously incorporates farm records, crop history, environmental conditions, prior conversations, and retrieved agricultural knowledge into its reasoning process. This enables context-aware interactions that more closely resemble consultations with a human agricultural expert.

The architecture of Mozhi consists of three major components: a language model integration layer responsible for communication with large language models, a hierarchical memory subsystem for long-term conversational persistence, and a contextual reasoning framework that dynamically assembles information from multiple NAVA modules before generating responses. Additionally, Mozhi incorporates automated note extraction mechanisms that convert conversational interactions into structured farm records. Together, these components provide the foundation for intelligent and context-sensitive agricultural assistance.

3.5.1 Llama-3 Chat Integration

The conversational capabilities of Mozhi are built upon large language models accessed through the Hugging Face Router API. Rather than embedding a language model directly within the application infrastructure, Mozhi utilizes a lightweight communication layer known as the `ChatClient`, which interacts with an OpenAI-compatible chat completion endpoint through standard HTTP requests. This design decouples the conversational subsystem from specific

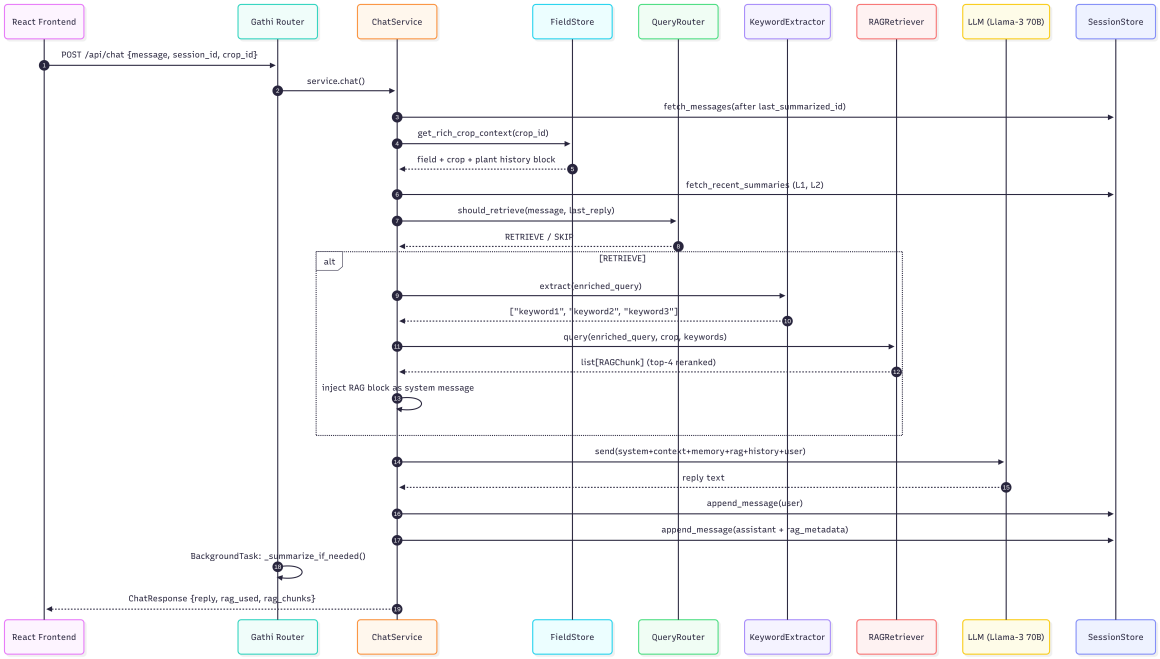

Figure 11: Workflow of Mozhi Pipeline

model implementations and allows model configurations to be modified without altering application logic.

A dual-model strategy is employed within the system. User-facing conversations are generated using Meta Llama-3.3 70B Instruct, which provides advanced reasoning capabilities and high-quality natural language generation. Internal operations such as summarization, memory consolidation, retrieval routing, keyword extraction, and note generation are delegated to the smaller Llama-3.1 8B Instruct model. This separation reduces computational costs and inference latency while preserving response quality for farmer interactions.

The language model is not used as a standalone chatbot. Instead, every user query passes through a structured orchestration process that assembles contextual information before invoking the model. The prompt architecture includes multiple layers of information, including system instructions, farm-specific context, conversational memory, retrieved agricultural references, recent dialogue history, and the current user query. This layered prompting strategy ensures that generated responses remain relevant to the farmer's specific circumstances and operational context.

The central orchestration mechanism is implemented within the `ChatService` component. Upon receiving a user message, the service resolves session information, retrieves prior conversation history, assembles contextual information, performs retrieval decisions when necessary, invokes the language model, and stores the resulting interaction. By centralizing these responsibilities, Mozhi maintains consistent conversational behavior while ensuring integration with the broader NAVA ecosystem.

A key design objective of the chat integration layer is to minimize hallucinated responses. Consequently, the language model is rarely asked to respond based solely on its pre-trained knowledge. Instead, contextual information from farm records and retrieved agricultural documents is supplied whenever available, allowing the model to generate responses grounded in verifiable data sources. This approach improves reliability and aligns conversational outputs with the current state of the farm being discussed.

3.5.2 Multi-Level Memory Hierarchy

One of the major challenges associated with large language model applications is the limitation imposed by finite context windows. As conversations become longer, retaining all previous messages within every prompt becomes computationally expensive and eventually infeasible. Mozhi addresses this challenge through a hierarchical memory architecture that preserves long-term conversational context while maintaining efficient prompt construction.

Conversation persistence is managed through the `SessionStore` subsystem, which utilizes SQLite for durable storage of messages, summaries, session metadata, and contextual associations. Each conversation session is assigned a unique identifier that links messages to specific farms and crops. This architecture enables users to maintain independent discussions across multiple agricultural contexts without information leakage between sessions.

The memory hierarchy consists of two distinct levels. The first level, referred to as Level-1 memory, captures recent conversational interactions. When the number of unsummarized messages exceeds a predefined threshold, a summarization process is triggered. A lightweight language model generates a concise summary representing the essential content of the interaction history. These summaries preserve both user intentions and system responses, allowing important information to remain available without retaining every original message.

As multiple Level-1 summaries accumulate, Mozhi performs a secondary consolidation process known as Level-2 rollup. During this stage, several older Level-1 summaries are combined into a single long-term memory representation. The resulting rollup captures broader conversational trends, decisions, and historical context while occupying significantly less prompt space than the original messages. This hierarchical compression strategy enables conversations to span extended periods without exceeding language model context limitations.

When constructing prompts, Mozhi injects both memory levels into the system context. Level-2 memory provides historical continuity by representing long-term discussions, while Level-1 memory supplies recent conversational details. Together, these memory layers allow the language model to maintain awareness of past interactions and avoid repetitive questioning or contradictory recommendations.

The multi-level memory architecture therefore addresses a fundamental limitation of conversational AI systems by providing scalable long-term memory while preserving computational

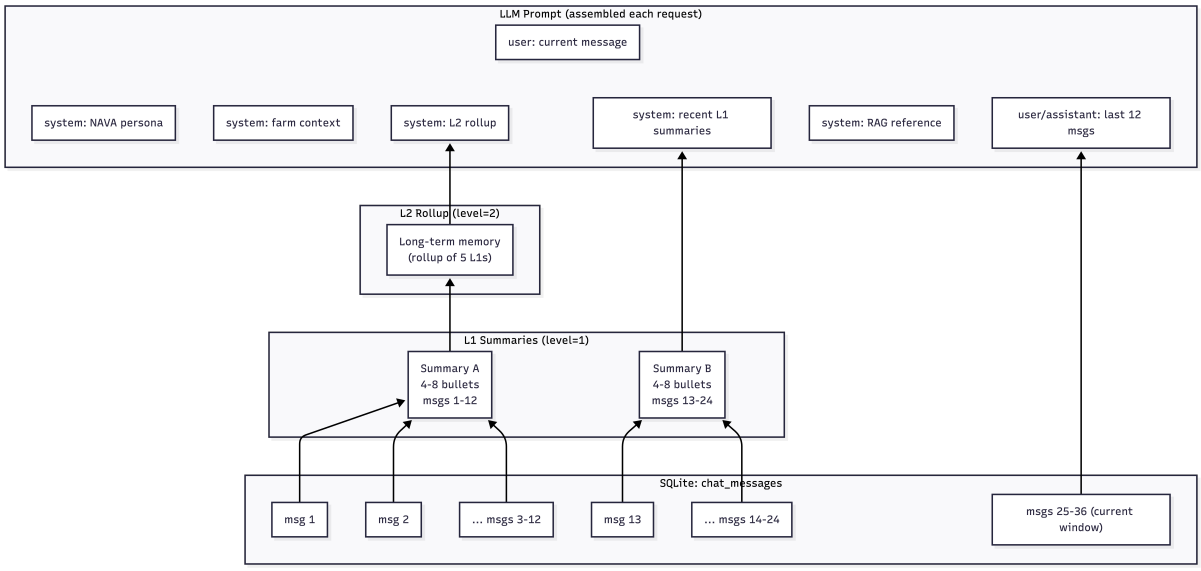

Figure 12: Hierarchical Memory: Messages, L1, and L2 Rollups

efficiency. As a result, Mozhi can support season-long agricultural consultations without losing awareness of earlier discussions or decisions.

3.5.3 Dynamic Context Injection (Farm, Crop, Weather)

Accurate agricultural advisory requires awareness of the specific farming environment under discussion. Generic responses generated without contextual information may fail to account for crop variety, environmental conditions, disease history, or ongoing management activities. To overcome this limitation, Mozhi employs a dynamic context injection framework that integrates farm-specific information directly into the language model prompt.

The context assembly process begins with retrieval of field-level information from the Shared Foundation Layer. This includes metadata describing farm location, cultivated area, soil characteristics, and associated crop records. When a conversation is linked to a specific crop, Mozhi additionally retrieves crop-level information such as crop variety, growth stage, cultivation season, and manually recorded notes. These records establish the baseline context for all subsequent reasoning processes.

Weather information is incorporated as an additional contextual layer. Rather than issuing external weather requests during conversation processing, Mozhi retrieves weather data directly from values previously stored within the farm database. Parameters such as temperature, humidity, precipitation, wind speed, and update timestamps are inserted into the prompt as part of the contextual representation. This design eliminates network latency during conversational inference while ensuring that responses remain aware of current environmental conditions.

Historical plant health records are also included in the context assembly process. Recent

disease diagnoses generated by Mizhi, together with VNIR monitoring results and stress alerts, are incorporated into the prompt structure. Disease detection records receive higher priority due to their direct diagnostic significance, while VNIR observations provide supporting information regarding physiological stress conditions. By integrating these records, Mozhi gains awareness of ongoing agricultural issues and can provide recommendations tailored to the actual health status of the crop.

The contextual framework additionally incorporates sibling crop information and farm-wide observations. This broader perspective enables the conversational system to reason about interactions between different cultivation areas and maintain awareness of overall farm conditions rather than focusing exclusively on isolated plants.

A notable design decision is that contextual information is injected as system-level instructions rather than user-visible content. Consequently, the language model utilizes the information implicitly when generating responses without repeatedly referencing database records or explicitly stating that information was retrieved from farm history. This produces more natural conversational behavior while maintaining strong contextual grounding.

Through dynamic context injection, Mozhi transforms the language model from a general-purpose conversational agent into a farm-aware advisory system capable of generating recommendations that reflect the current state of a user's agricultural environment.

3.5.4 Automated Agronomic Note Extraction

Agricultural conversations often contain valuable operational information regarding management decisions, treatments applied, observed symptoms, and corrective actions performed by farmers. Such information is frequently lost when conversational systems treat interactions as transient exchanges. Mozhi addresses this challenge through an automated agronomic note extraction mechanism that converts relevant conversational content into structured farm records.

The note extraction process is triggered after the generation of Level-1 conversation summaries. Once a summary becomes available, it is analyzed by a secondary language model using a specialized extraction prompt. Rather than generating a conventional response, the model is instructed to identify concrete actions and decisions made by the farmer during the conversation. Examples include pesticide applications, fungicide treatments, irrigation adjustments, crop removal activities, fertilization decisions, and other management interventions.

Only explicit and actionable agronomic activities are retained. General discussion, speculative statements, and ambiguous references are excluded from the extraction process to maintain the quality of stored records. The extracted information is subsequently appended to the crop-specific notes maintained within the agricultural database. These entries are timestamped and organized under a dedicated section reserved for automatically generated observations.

The resulting notes become part of the contextual information available during future conversa-

tions. Consequently, decisions recorded in previous interactions influence subsequent advisory responses, enabling the system to maintain continuity across extended cultivation periods. For example, if a farmer previously reported applying a specific fungicide, future recommendations can account for that action rather than suggesting redundant treatments.

This capability effectively transforms conversational interactions into a continuously evolving agricultural record. Rather than functioning solely as a communication interface, Mozhi becomes a mechanism for capturing and preserving operational knowledge generated during day-to-day farm management activities. Such persistence strengthens contextual awareness, improves advisory consistency, and contributes to the creation of a comprehensive digital history for each crop under management.

Overall, the integration of conversational intelligence, hierarchical memory management, dynamic contextual grounding, and automated note extraction enables Mozhi to function as the cognitive core of the NAVA platform. By maintaining awareness of farm history, environmental conditions, and previous interactions, the module supports informed and contextually relevant agricultural assistance throughout the cultivation lifecycle.

3.6 Yukthi: Knowledge Retrieval Pipeline

Yukthi serves as the knowledge layer of the NAVA platform and implements the Retrieval-Augmented Generation (RAG) framework that supports evidence-grounded agricultural advisory. While large language models possess extensive general knowledge, they may generate inaccurate or unverifiable recommendations when responding to domain-specific agricultural queries. Such inaccuracies can be particularly problematic in farming contexts where incorrect advice regarding disease management, pesticide application, fertilization, or cultivation practices may result in economic losses or environmental consequences.

To address this challenge, Yukthi was designed as a dedicated knowledge retrieval subsystem that grounds language model responses in authoritative agricultural reference materials. The module transforms agricultural extension documents into searchable vector representations, retrieves relevant passages at query time, and supplies verified contextual information to the conversational layer before response generation. Through this architecture, Yukthi enables NAVA to provide recommendations that are aligned with documented agricultural practices rather than relying solely on the parametric knowledge of the language model.

The retrieval pipeline consists of four major stages: document ingestion and chunking, query routing, agronomic keyword extraction, and hybrid retrieval. These stages collectively ensure that relevant information is identified efficiently and incorporated into conversational responses when required.

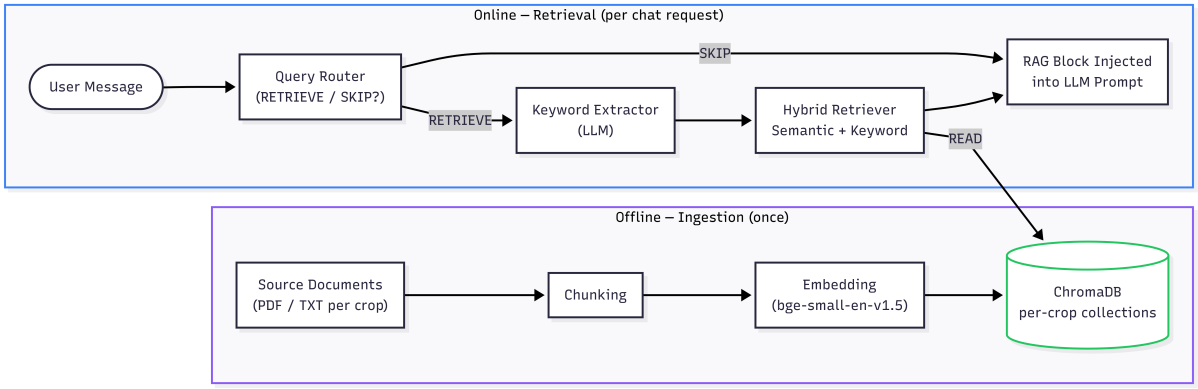

Figure 13: High-Level Architecture of Yukthi RAG Pipeline

3.6.1 Document Ingestion and Chunking

The first stage of the Yukthi pipeline is responsible for transforming agricultural documents into a structured representation suitable for semantic retrieval. Agricultural knowledge sources are organized according to crop type, with each crop maintaining its own collection of reference documents. These sources include agricultural extension manuals, disease management guides, cultivation recommendations, and other verified agronomic resources. Supported document formats include plain text files and Portable Document Format (PDF) documents.

The ingestion process is implemented through the `RAGPipeline` component, which executes a sequence of source discovery, text extraction, chunk generation, embedding creation, and database insertion operations. During source discovery, all eligible documents associated with a particular crop are identified automatically. This design enables straightforward expansion of the knowledge base through the addition of new reference materials without requiring modifications to retrieval logic.

Text extraction and chunking constitute the most critical preprocessing stage. Large documents cannot be embedded and retrieved efficiently as single units because relevant information is often localized within specific sections. Consequently, documents are divided into smaller semantic chunks that preserve contextual coherence while improving retrieval granularity. For text-based sources, segmentation is performed using paragraph boundaries, with additional splitting applied when content exceeds predefined size constraints. PDF documents are first processed using the PyMuPDF library to extract textual content before undergoing the same chunking procedure.

During chunk generation, the system attempts to identify section headers and preserve them as metadata. Each chunk therefore contains not only textual content but also information regarding its source document, section title, crop association, and positional index. This metadata supports source attribution and facilitates more meaningful retrieval results during subsequent stages.

Following chunk creation, semantic embeddings are generated using the *BAAI/bge-small-en-*

v1.5 sentence embedding model through the Sentence Transformers framework. This model produces 384-dimensional dense vector representations optimized for semantic retrieval tasks. Unlike the language models used within Mozhi, embedding generation is performed entirely locally and does not require external API calls. The generated embeddings capture semantic relationships between agricultural concepts, enabling retrieval based on meaning rather than exact keyword matching.

The resulting embeddings and metadata are stored within ChromaDB collections. Each crop is assigned an independent collection, thereby preventing retrieval contamination between unrelated crops. This organization improves retrieval precision and ensures that crop-specific queries are matched only against relevant agronomic knowledge sources.

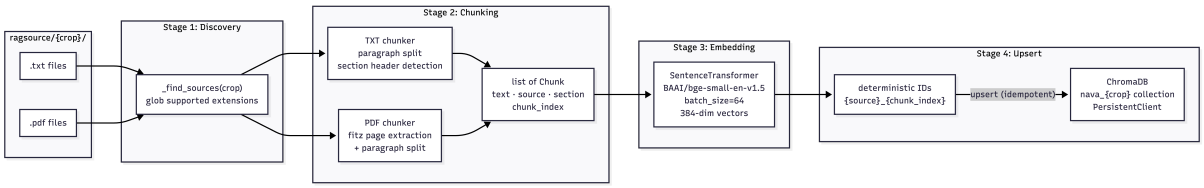

Figure 14: Document Ingestion Workflows

3.6.2 Query Routing Classifier

Not all conversational queries require retrieval from the knowledge base. Messages such as greetings, acknowledgments, conversational follow-ups, or references to information already discussed can often be answered using existing conversational context. Performing retrieval for every interaction would increase latency, consume computational resources, and potentially introduce irrelevant information into the prompt. To address this issue, Yukthi incorporates a query routing classifier that determines whether retrieval should be executed for a given message.

The routing mechanism is implemented through the `QueryRouter` component. Rather than relying on rule-based heuristics, the router uses *Meta-Llama-3.1-8B-Instruct* as a lightweight classification model. For each incoming query, the router receives the current user message together with the most recent assistant response and evaluates whether external knowledge retrieval is required. The model is prompted to generate only one of two outputs: `RETRIEVE` or `SKIP`.

Queries requesting disease treatment recommendations, pesticide information, fertilization guidance, cultivation practices, irrigation strategies, crop management advice, or agronomic explanations are typically classified as retrieval candidates. In contrast, conversational messages, acknowledgments, clarifications, and follow-up interactions that can be resolved using existing context are generally routed through the skip path.

An important advantage of the routing approach is that the classifier considers conversational context rather than analyzing messages in isolation. Consequently, ambiguous follow-up questions such as "What dosage should I use?" can be interpreted correctly based on the preceding discussion. This contextual awareness improves routing accuracy and prevents unnecessary retrieval operations.

The routing stage therefore acts as an intelligent gatekeeper that balances retrieval effectiveness against computational efficiency. By selectively invoking the retrieval pipeline only when necessary, Yukthi reduces latency while preserving access to verified agricultural knowledge whenever it is needed.

3.6.3 Agronomic Keyword Extraction

Once a query has been identified as requiring retrieval, the next challenge is constructing an effective search representation. User queries are often vague, incomplete, or expressed in conversational language. Directly embedding such queries may fail to capture the specific agronomic concepts required for precise retrieval. To address this limitation, Yukthi incorporates a dedicated keyword extraction stage.

Keyword extraction is performed by the `KeywordExtractor` component using *Meta-Llama-3.1-8B-Instruct*. The model receives an enriched retrieval query and is instructed to generate a concise set of highly relevant agronomic search terms. Typically, three keywords or short phrases are extracted, representing the most important agricultural concepts present within the query.

The retrieval query supplied to the extractor is not limited to the user's original message. Prior to keyword extraction, Mozhi enriches the query using contextual information obtained from farm records. Crop names, disease diagnoses, and stress monitoring results may be incorporated into the retrieval query to provide additional semantic specificity. For example, a vague request concerning treatment recommendations can be transformed into a context-aware retrieval query containing the crop type and detected disease condition.

The extracted keywords are subsequently used during retrieval to identify passages containing agriculturally significant terminology that may not rank highly through semantic similarity alone. Examples include disease names, pesticide compounds, nutrient deficiencies, cultivation techniques, and crop-specific management practices. By explicitly identifying such concepts, the keyword extraction stage improves retrieval precision for highly specialized agricultural queries.

To improve robustness, Yukthi also implements a fallback mechanism. If keyword extraction fails or produces no valid output, heuristic term extraction is performed directly from the retrieval query. This ensures that the retrieval process remains operational even when language model outputs are unavailable or malformed.

The use of *Meta-Llama-3.1-8B-Instruct* for keyword extraction represents an important design choice. Unlike rule-based keyword extraction methods, the language model can identify agriculturally meaningful concepts even when they are expressed indirectly or embedded within natural language descriptions. This capability enhances retrieval effectiveness in real-world conversational scenarios.

3.6.4 Hybrid Retrieval Strategy (Semantic and Keyword)

The final stage of the Yukthi pipeline is the retrieval process itself. A purely semantic retrieval strategy may overlook highly specific agricultural terms, while a purely keyword-based approach may fail to capture broader conceptual relationships. To overcome these limitations, Yukthi employs a hybrid retrieval strategy that combines semantic similarity search with keyword-guided retrieval.

The retrieval process begins by converting the enriched query into a dense vector representation using the same *BAAI/bge-small-en-v1.5* embedding model used during document ingestion. This consistency ensures that query vectors and document vectors occupy the same semantic space, enabling effective similarity comparison.

The first retrieval stage performs semantic nearest-neighbor search within the appropriate ChromaDB crop collection. Candidate passages are selected according to vector similarity, allowing the system to retrieve information that is conceptually related to the query even when identical keywords are absent. This capability is particularly useful for natural language questions that may be phrased differently from the source documents.

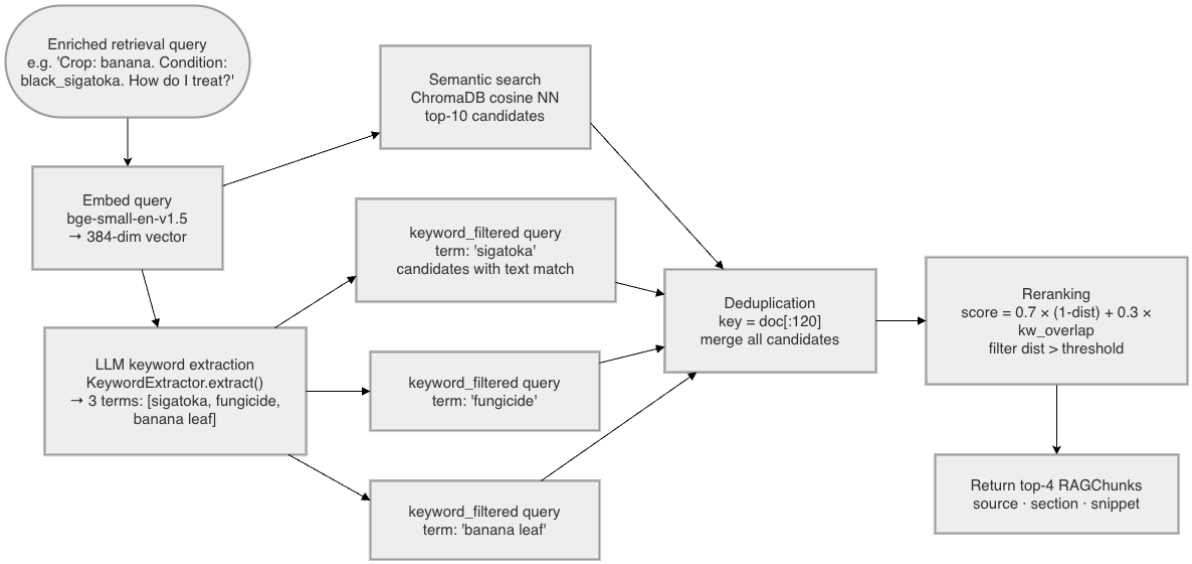

Figure 15: Hybrid Retrieval Workflow

The second retrieval stage utilizes the agronomic keywords generated by *Meta-Llama-3.1-8B-*

Instruct. For each extracted keyword, ChromaDB executes keyword-filtered semantic searches that restrict retrieval to documents containing the specified term. This strategy enables the system to identify passages that may contain highly relevant domain-specific information even if their semantic similarity scores are lower than those of other candidates.

Results from semantic retrieval and keyword-filtered retrieval are subsequently merged and deduplicated. Candidate passages are then subjected to a reranking procedure that combines semantic similarity scores with keyword overlap measures. The reranking mechanism prioritizes passages that are both semantically relevant and strongly aligned with the extracted agronomic concepts. This multi-stage selection process improves retrieval precision while preserving broad semantic coverage.

Following reranking, the highest-ranked passages are returned as structured retrieval objects containing passage text, source information, section metadata, and relevance scores. These retrieved passages are injected into the conversational prompt as verified reference material before the final response is generated by *Meta-Llama-3.3-70B-Instruct* within the Mozhi module. Consequently, the language model produces responses grounded in authoritative agricultural documentation rather than relying exclusively on its pre-trained knowledge.

The hybrid retrieval architecture therefore represents a balance between semantic understanding and explicit domain terminology matching. By integrating vector similarity search, LLM-driven keyword extraction, keyword-constrained retrieval, and reranking, Yukthi provides a robust mechanism for identifying relevant agricultural knowledge and supplying evidence-based context to the conversational intelligence layer of NAVA.

3.7 Shared Foundation Layer

The Shared Foundation Layer serves as the infrastructural backbone of the NAVA platform and provides the common services upon which all higher-level modules depend. Unlike Gathi, Mizhi, Mozhi, and Yukthi, which implement domain-specific functionality, the Shared Foundation Layer is responsible for cross-cutting concerns including configuration management, persistent data storage, geospatial processing, weather integration, schema definitions, and utility services. Its primary purpose is to establish a consistent and reusable foundation that supports communication, data persistence, and contextual information management throughout the system.

A key architectural principle adopted within NAVA is the separation of infrastructure from business logic. Consequently, all domain modules depend on the Shared Foundation Layer, while the Shared module remains independent of higher-level components. This unidirectional dependency structure improves modularity, reduces coupling, and simplifies testing and maintenance activities.

The Shared Foundation Layer consists of three principal capabilities relevant to the operation of

NAVA: localized SQLite-based storage infrastructure, geospatial coordinate resolution services, and environmental context acquisition through weather integration. These services collectively provide the contextual and persistent information required by the perception, retrieval, and conversational intelligence modules.

3.7.1 Localized SQLite Storage (Field and Session Stores)

Persistent data management within NAVA is implemented using SQLite. The choice of SQLite was motivated by the project's objective of providing a lightweight, self-contained agricultural intelligence platform that does not require external database servers or cloud-hosted storage infrastructure. This design simplifies deployment while ensuring that all user information, farm records, conversational histories, and monitoring results remain locally manageable.

The storage architecture is organized into three primary persistence layers. The first layer consists of a global user registry database responsible for authentication and session management. The second layer contains per-user farm databases that store agricultural records and conversational information. The third layer consists of the ChromaDB vector store used by Yukthi for knowledge retrieval operations. Although ChromaDB serves a retrieval function, it forms part of the overall persistence architecture of the platform.

A notable design decision is the adoption of a two-database SQLite architecture. Instead of storing all information within a single shared database, NAVA separates user identity management from farm operational data. The global database stores user accounts, authentication credentials, and session tokens, while each registered user receives a dedicated SQLite database containing fields, crops, plants, monitoring records, and chat histories. This separation improves data isolation and simplifies account management operations.

The global user registry is managed through the `UserStore` component. User credentials are stored using bcrypt password hashing, ensuring that plaintext passwords are never persisted. Authentication sessions are represented through cryptographically generated tokens, which are stored together with expiration metadata. During authentication, session validation is performed by matching tokens against stored records and verifying expiration constraints. This mechanism forms the foundation of the platform's access control infrastructure.

Farm-specific information is managed through the `FieldStore` component. The data model follows a hierarchical structure consisting of fields, crops, plants, events, and VNIR history records. Fields represent physical agricultural locations, crops represent cultivated species within those fields, and plants provide fine-grained monitoring units. Disease diagnosis results and VNIR analyses are recorded as event entries, while VNIR ratios are additionally maintained as a dedicated time-series dataset to support efficient stress monitoring calculations.

The events table serves as a unified historical repository for both disease classification and stress monitoring operations. Diagnostic outputs generated by Mizhi are stored as structured

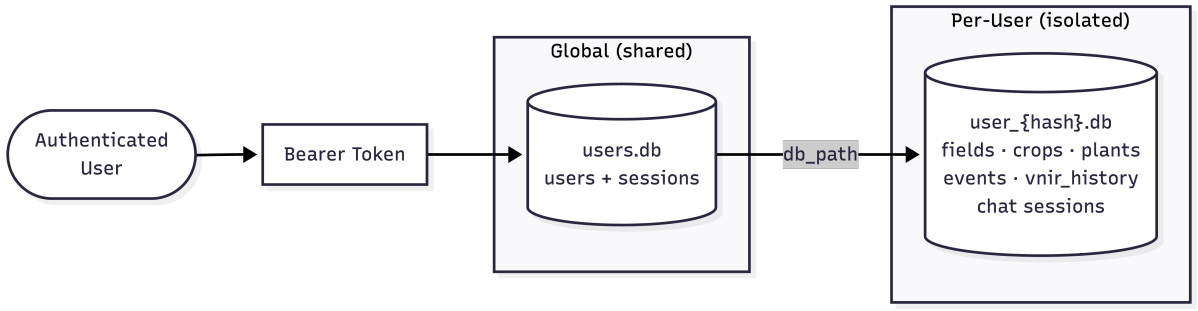

Figure 16: Two-Database Architecture

JSON payloads together with associated metadata such as timestamps, crop identifiers, and plant references. This design provides traceability and enables historical context generation for conversational interactions.

The storage subsystem also supports conversational persistence. Chat messages, summaries, contextual associations, and memory structures generated by Mozhi are stored within dedicated chat tables residing in the user's farm database. This design allows conversational memory to remain closely coupled with the agricultural records to which it relates. Consequently, chat sessions can be reconstructed and summarized across extended periods while preserving farm-specific context.

An additional feature of the storage architecture is the automatic generation of contextual summaries. The `FieldStore` dynamically synthesizes field-level and crop-level contextual descriptions from stored records. These summaries aggregate information such as crop varieties, growth stages, disease histories, and stress monitoring results. The generated context is subsequently injected into Mozhi's conversational prompts, enabling context-aware dialogue generation.

To support long-term maintainability, the storage layer incorporates a schema migration mechanism. Rather than relying on an external migration framework, database structures are inspected during initialization and updated incrementally when required. New columns and tables can therefore be introduced without disrupting existing user databases, ensuring backward compatibility across software updates.

Table 6: SQLite Tables and Their Functional Roles in NAVA

Table Name	**Module**	**Primary Role**	**Functional Purpose**
users	Shared	Global user registry	Stores credentials, profile data, onboarding status, and per-user DB path
sessions	Shared	Session/token management	Maintains active login tokens with expiry validation
fields	Gathi	Farm-level metadata storage	Stores location, soil, weather, notes, and AI-generated shared context
crops	Gathi	Crop lifecycle tracking	Maintains crop type, stage, season, and agronomic notes
plants	Gathi	Plant-level tracking	Stores individual plant records for monitoring and diagnosis
events	Gathi	Unified activity log	Stores diagnosis and VNIR event payloads as JSON
vnir_history	Gathi	Temporal VNIR monitoring	Stores ratio timeseries for rolling-average and stress trend analysis
chat_messages	Mozhi	Chat history persistence	Stores all user-assistant conversations with RAG metadata
chat_state	Mozhi	Summarisation checkpointing	Tracks message summarisation progress for bounded context
chat_summaries	Mozhi	Hierarchical memory storage	Stores Level-1 and Level-2 summaries for long-term memory
chat_context	Mozhi	Session-to-farm mapping	Links conversations to specific fields and crops for contextual AI

3.7.2 Geo-Coordinate Resolution

Geospatial awareness forms an important component of contextual agricultural intelligence. Many advisory recommendations depend on environmental conditions and geographic location. To support location-aware services, NAVA incorporates a geospatial resolution subsystem within the Shared Foundation Layer.

The geospatial functionality is implemented through the `geo_context.py` utility module. Its primary responsibility is to convert user-provided location descriptions into geographic coordinates that can subsequently be used for environmental data retrieval and spatial contextualization.

When a new field is created, the user may provide either a textual location description or explicit geographic coordinates. The system first attempts to determine whether the supplied value already represents latitude and longitude coordinates. If valid coordinates are detected, they are parsed directly and stored without external service invocation. This shortcut minimizes unnecessary network requests and improves efficiency.

For textual location descriptions, the system utilizes the OpenStreetMap Nominatim geocoding service. The provided location string is submitted to the geocoding API, which returns corresponding latitude and longitude values. The first matching result is selected and stored within the field record as persistent geospatial metadata. By persisting coordinates after the initial lookup, the system avoids repeated geocoding requests for subsequent operations involving the same field.

The geocoding process is executed asynchronously following field creation or modification. This design prevents geospatial processing delays from affecting user interactions. Fields therefore become immediately available within the application while location enrichment continues in the background.

Once geographic coordinates have been resolved, they become part of the permanent field representation stored within the database. These coordinates are subsequently reused by weather retrieval services and contextual reasoning mechanisms without requiring additional geocoding operations. The resulting workflow improves efficiency while reducing dependence on external geospatial services.

3.7.3 Open-Meteo Weather Integration

Weather conditions represent one of the most influential environmental factors affecting agricultural productivity. Temperature, humidity, precipitation, and wind conditions directly influence disease development, irrigation requirements, and crop growth dynamics. Consequently, NAVA incorporates weather integration as a foundational contextual service.

Weather acquisition is implemented using the Open-Meteo API. Once field coordinates have been resolved through the geospatial subsystem, the weather service retrieves current environ-

mental measurements corresponding to the field location. The retrieved parameters include temperature, relative humidity, precipitation, and wind speed. These measurements provide a concise representation of current environmental conditions relevant to agricultural decision-making.

Unlike systems that perform weather lookups during every user interaction, NAVA adopts a persistent weather storage strategy. Retrieved weather values are written directly into the field database and stored together with update timestamps. As a result, conversational reasoning and dashboard operations can access weather information locally without initiating additional external API requests. This approach reduces latency and minimizes dependence on network availability during routine interactions.

Weather information is refreshed through multiple mechanisms. Initial weather acquisition occurs when a field is first created and successfully geocoded. Additional refresh operations are triggered when users authenticate, ensuring that environmental information remains reasonably current. Manual refresh functionality is also supported, allowing users to request updated weather data when necessary.

The weather subsystem plays a significant role within Mozhi's context injection framework. Stored environmental measurements are incorporated into conversational prompts and become part of the contextual information supplied to *Meta-Llama-3.3-70B-Instruct* during response generation. Consequently, agricultural recommendations can be informed by current environmental conditions without requiring direct weather-related queries from the user.

Furthermore, the weather infrastructure contributes to farm-level contextual awareness maintained within the database. Weather information becomes part of the field context summaries generated by the Shared Foundation Layer and is therefore available to both conversational reasoning and future analytical extensions of the platform.

In summary, the Shared Foundation Layer provides the persistent storage, geospatial intelligence, and environmental context services required by all higher-level NAVA modules. Through its SQLite-based storage architecture, coordinate resolution capabilities, and weather integration framework, it establishes the infrastructural foundation upon which perception, retrieval, conversational intelligence, and farm management functionalities are built.

4 Experimental Results and Analysis

4.1 Introduction

The experimental evaluation of the NAVA (Next-Gen Agricultural Virtual Assistant) platform was designed to assess both the performance of the underlying machine learning models and the effectiveness of the integrated multi-modal agricultural intelligence system. Since NAVA combines computer vision, virtual near-infrared estimation, retrieval-augmented generation, conversational artificial intelligence, contextual memory management, and farm-aware reasoning within a unified framework, the evaluation process extends beyond conventional model benchmarking and includes validation of end-to-end system behavior.

The experimental methodology was divided into two complementary components. The first component focused on quantitative evaluation of the machine learning models responsible for crop disease classification and VNIR image synthesis. Standard performance metrics were used to measure predictive accuracy, classification effectiveness, and image reconstruction quality. The second component focused on qualitative evaluation of the integrated platform, examining how individual modules interact during realistic agricultural workflows. This included assessment of explainable disease diagnosis, stress monitoring, contextual memory management, retrieval-augmented reasoning, and knowledge-grounded conversational responses.

Unlike traditional machine learning projects that evaluate a single predictive model, NAVA operates as a complete agricultural intelligence ecosystem. Consequently, evaluation was performed at both the component level and the system level to provide a comprehensive understanding of platform performance, reliability, and operational behavior. The results presented in this chapter are organized according to these evaluation objectives and form the basis for the analysis discussed in subsequent sections.

4.2 Testing Methodology and Setup

The testing methodology adopted for NAVA was designed to evaluate both model-specific performance and integrated system functionality. Quantitative experiments were conducted during the development and training of the core machine learning models, while qualitative evaluations were performed on the deployed platform using automated testing workflows that simulated realistic user interactions.

The qualitative evaluation framework focused on validating system behavior, contextual awareness, retrieval effectiveness, explainability, and memory management. In contrast, the quantitative evaluation framework assessed the predictive performance of the disease classification and VNIR estimation models using established machine learning evaluation metrics. Together, these methodologies enabled a comprehensive assessment of the platform's technical capabilities.

4.2.1 Qualitative Testing Framework

The qualitative evaluation of NAVA was conducted using a dedicated automated testing framework implemented within the project's testing environment. The objective of this framework was to validate the correctness of interactions between the major system modules and to examine the behavior of the platform under realistic agricultural usage scenarios. Rather than measuring numerical performance alone, these evaluations focused on understanding how information flows through the system and how different components contribute to the final outputs generated for users.

The testing framework generated structured markdown reports that documented each stage of processing, including user inputs, intermediate outputs, routing decisions, contextual information, retrieved knowledge, and final responses. Four major categories of qualitative evaluation were performed.

The first category evaluated the disease diagnosis workflow. The generated disease evaluation reports documented the true disease label, predicted disease category, confidence score, and reliability status produced by the EfficientNet-B0 classification pipeline. The reports additionally included the corresponding Grad-CAM visualizations, enabling inspection of the image regions that influenced the model's predictions. This evaluation provided qualitative insight into both classification behavior and model explainability.

The second category evaluated the VNIR monitoring subsystem. These reports examined the complete monitoring pipeline, including leaf isolation, VNIR estimation, baseline calibration, and stress detection. For each observation, the reports recorded the generated VNIR representation, calculated VNIR ratio, historical comparison statistics, and stress assessment status. Particular attention was given to validating the transition between calibration, warning, and critical monitoring states during sequential observations.

The third category evaluated conversational intelligence and memory management. These tests examined the behavior of Mozhi by recording user queries, routing decisions, generated prompts, injected contextual information, memory summaries, and extracted agronomic notes. The evaluation verified that conversation histories were correctly summarized, that contextual information was preserved across extended interactions, and that automatically generated notes were successfully integrated into future conversations.

The fourth category focused on retrieval-augmented generation. This evaluation was structured as a comparative study in which conversational responses generated with retrieval augmentation were compared against responses generated without access to external agricultural knowledge. The reports documented the retrieved knowledge chunks, source documents, section references, and associated snippets used during response generation. This process enabled qualitative assessment of the contribution of the Yukthi retrieval pipeline to factual grounding and domain specificity.

Collectively, these automated testing workflows provided a systematic mechanism for validating the integrated behavior of the NAVA platform. The generated reports served as diagnostic artifacts that enabled detailed examination of perception, monitoring, retrieval, reasoning, and memory-management processes across the complete agricultural intelligence pipeline.

4.2.2 Evaluation Criteria

The quantitative evaluation focused on the machine learning models that form the perception layer of the NAVA platform. Separate evaluation criteria were employed for disease classification and VNIR estimation in accordance with the objectives of each task. The experiments were conducted using dedicated training and validation pipelines implemented during model development.

Disease Classification Evaluation Metrics: The disease classification study involved comparative evaluation of three convolutional neural network architectures: MobileNetV2, ResNet50, and EfficientNet-B0. The objective of this comparison was to identify the architecture that provided the most suitable balance between predictive performance and deployment efficiency for agricultural disease diagnosis.

All candidate models were trained and evaluated using identical dataset partitions and preprocessing procedures to ensure fair comparison. Performance was assessed using a combination of training metrics and classification metrics commonly employed in multi-class image classification tasks.

The evaluation criteria included:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss
- Per-Class Accuracy
- Precision
- Recall
- F1-Score
- Overall Classification Accuracy

Training and validation accuracy were used to assess convergence behavior and generalization capability throughout the training process. Corresponding loss values were analyzed to evaluate optimization stability and detect potential overfitting or underfitting behavior.

Per-class accuracy analysis was performed to examine classification performance across individual disease categories and crop species. Since the dataset contained multiple crops and disease classes, this metric was particularly important for identifying class-specific strengths and weaknesses.

Precision, recall, and F1-score were used to provide a more comprehensive assessment of classification effectiveness. Precision measured the correctness of positive predictions, recall evaluated the ability to identify true disease instances, and F1-score provided a balanced measure that incorporates both criteria.

Following comparative evaluation, EfficientNet-B0 demonstrated superior overall performance relative to MobileNetV2 and ResNet50 and was therefore selected as the final disease classification model integrated into the NAVA platform. The detailed comparative results are presented in Section 4.3.

VNIR Image Translation Evaluation Metrics: The VNIR estimation model was evaluated using image reconstruction quality metrics appropriate for image-to-image translation tasks. Since the objective of the Thanal subsystem is to generate virtual near-infrared representations from RGB imagery, evaluation focused on measuring similarity between the generated VNIR images and their corresponding reference targets.

Two standard image quality metrics were employed:

- Peak Signal-to-Noise Ratio (PSNR)
- Structural Similarity Index Measure (SSIM)

PSNR quantifies reconstruction fidelity by measuring the difference between generated images and reference images. Higher PSNR values indicate lower reconstruction error and improved reconstruction quality.

SSIM evaluates perceptual similarity by considering structural information, luminance characteristics, and contrast relationships between images. Unlike purely pixel-based error measures, SSIM provides a more meaningful assessment of visual similarity and structural preservation.

Together, these metrics provided an objective assessment of the VNIR model's ability to reconstruct near-infrared information from RGB inputs and served as the primary criteria for evaluating the effectiveness of the Thanal image translation framework.

The combination of qualitative system-level evaluations and quantitative model-level metrics enabled comprehensive assessment of the NAVA platform. While quantitative experiments measured predictive and reconstruction performance, qualitative evaluations validated the be-

havior of the integrated agricultural intelligence ecosystem, providing a holistic framework for experimental analysis.

4.3 Quantitative Analysis

4.3.1 Vision Model Training, Comparison and Validation

The primary objective of the disease classification component was to identify a convolutional neural network architecture capable of delivering high diagnostic accuracy while maintaining computational efficiency suitable for deployment within the NAVA platform. To achieve this, a comparative evaluation was conducted using three widely adopted image classification architectures: MobileNetV2, ResNet-50, and EfficientNet-B0. All models were trained and evaluated under identical experimental conditions to ensure a fair comparison.

The training dataset was constructed by aggregating multiple publicly available agricultural disease datasets, including PlantVillage, PlantDoc, PaddyDoctor, and other crop disease repositories. To reduce class imbalance and improve training stability, a filtering strategy was applied whereby classes containing fewer than 300 samples were excluded, while classes containing more than 700 samples were downsampled. Following preprocessing, the final dataset consisted of 20,400 training and validation samples together with a held-out test set of 4,089 images spanning 34 disease categories across seven crop species.

The selected crop categories included banana, cassava, corn, cucumber, rice, soybean, and tomato. Across these crops, the dataset incorporated a diverse collection of fungal, bacterial, and viral diseases as well as healthy plant samples, enabling evaluation under realistic multi-class classification conditions.

To identify the most suitable architecture for deployment, each model was evaluated using overall classification accuracy, precision, recall, and F1-score. The comparative results are summarized in Table 7.

Table 7: Comparative Performance of MobileNetV2, ResNet-50, and EfficientNet-B0

Metric	**MobileNetV2**	**ResNet-50**	**EfficientNet-B0**
Accuracy	83.53%	85.39%	91.44%
Precision	0.8412	0.8605	0.92
Recall	0.8320	0.8490	0.92
F1-Score	0.8365	0.8547	0.92

The experimental results demonstrated clear performance differences among the evaluated architectures. MobileNetV2 achieved an overall test accuracy of 83.53%, with precision, recall, and F1-score values of 0.8412, 0.8320, and 0.8365 respectively. ResNet-50 produced improved performance, achieving 85.39% accuracy together with precision, recall, and F1-score

values of 0.8605, 0.8490, and 0.8547. However, EfficientNet-B0 outperformed both competing architectures, achieving a test accuracy of 91.44% and macro-averaged precision, recall, and F1-score values of 0.92.

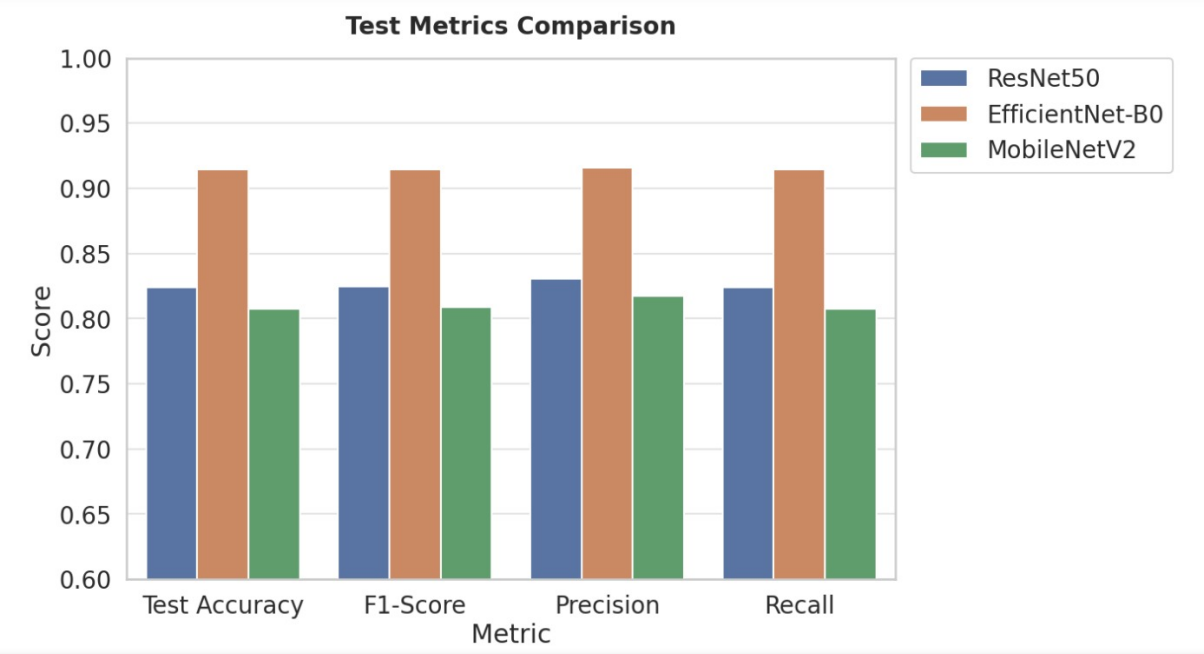

Figure 17: Comparative Performance of MobileNetV2, ResNet-50, and EfficientNet-B0

The superior performance of EfficientNet-B0 can be attributed to its compound scaling strategy, which simultaneously balances network depth, width, and input resolution. Unlike ResNet-50, which primarily improves representational capacity through increased depth, or MobileNetV2, which emphasizes lightweight computation through depthwise separable convolutions, EfficientNet-B0 provides a more balanced feature extraction framework capable of capturing both global structural abnormalities and localized lesion patterns. This characteristic is particularly beneficial in agricultural disease classification, where subtle texture variations often distinguish visually similar disease categories.

Based on the comparative evaluation, EfficientNet-B0 was selected as the final disease classification model integrated into the Mizhi perception pipeline. The selection was motivated not only by its superior classification performance but also by its parameter efficiency and suitability for practical deployment scenarios.

Following model selection, a detailed evaluation of EfficientNet-B0 was conducted using the held-out test dataset. The model achieved an overall classification accuracy of 91.44%, indicating that more than nine out of every ten test images were correctly classified across the 34 disease categories. The macro-averaged precision score of 92.00% demonstrated that predicted disease labels were highly reliable, while the macro-averaged recall score of 92.00% indicated strong sensitivity in identifying true disease instances. The resulting F1-score of 92.00% confirmed balanced classification performance across the diverse crop and disease categories represented

within the dataset.

To further investigate performance consistency across crops, aggregated crop-level accuracy analysis was performed. The results showed strong performance across all evaluated crop groups, with soybean, banana, and rice exhibiting the highest classification accuracies. Although certain categories such as tomato and cucumber exhibited comparatively lower performance, overall classification effectiveness remained consistently high across all crop groups.

Class-wise analysis revealed that diseases such as Banana Sigatoka, Rice Bacterial Leaf Streak, Soybean Downy Mildew, Corn Common Rust, and Rice Bacterial Panicle Blight achieved near-perfect classification performance. Conversely, relatively lower accuracies were observed for classes including Cassava Blight, Tomato Bacterial Leaf Spot, Tomato Septoria Leaf Spot, Corn Cercospora Leaf Spot, and Healthy Cucumber leaves. These results suggest that visually similar symptoms and intra-class variability remain challenging factors in agricultural image classification.

Analysis of the confusion matrix further revealed that the majority of classification errors occurred between diseases exhibiting similar visual symptoms. Examples included confusion between Corn Cercospora Leaf Spot and Northern Leaf Blight, as well as confusion among several tomato disease categories. Such observations are consistent with the inherent visual similarity of these diseases and highlight potential directions for future dataset expansion and model refinement.

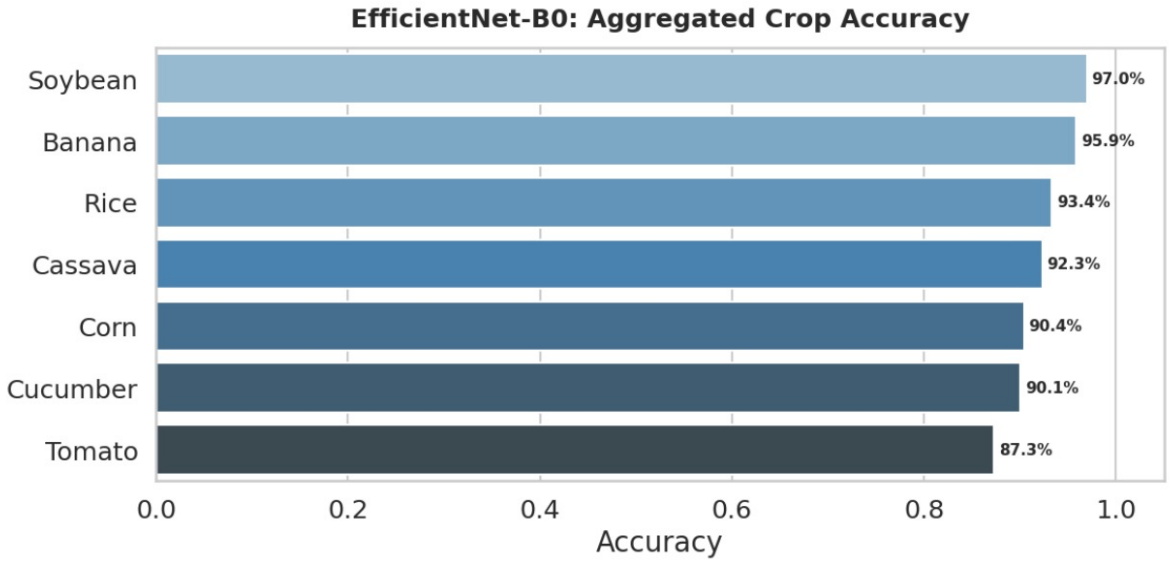

Figure 18: Aggregated Crop-Level Classification Accuracy

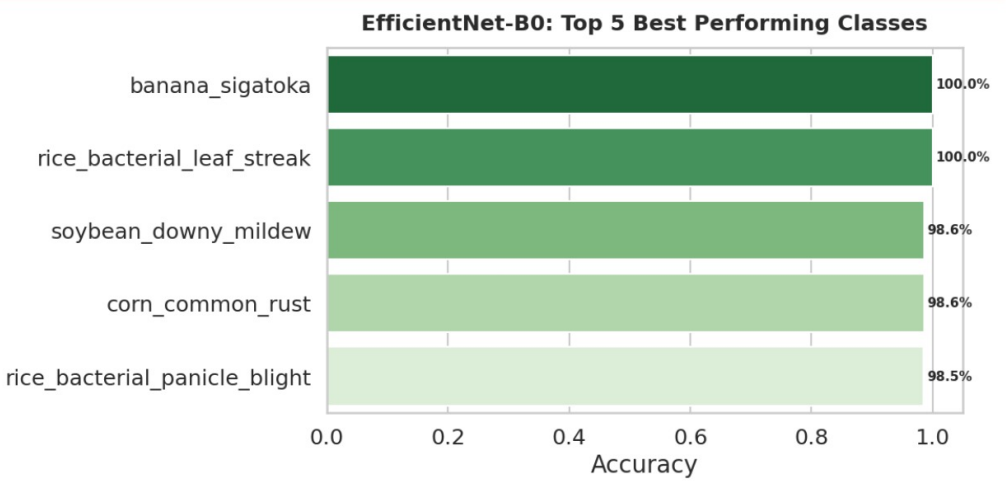

Figure 19: Top Performing Disease Classes

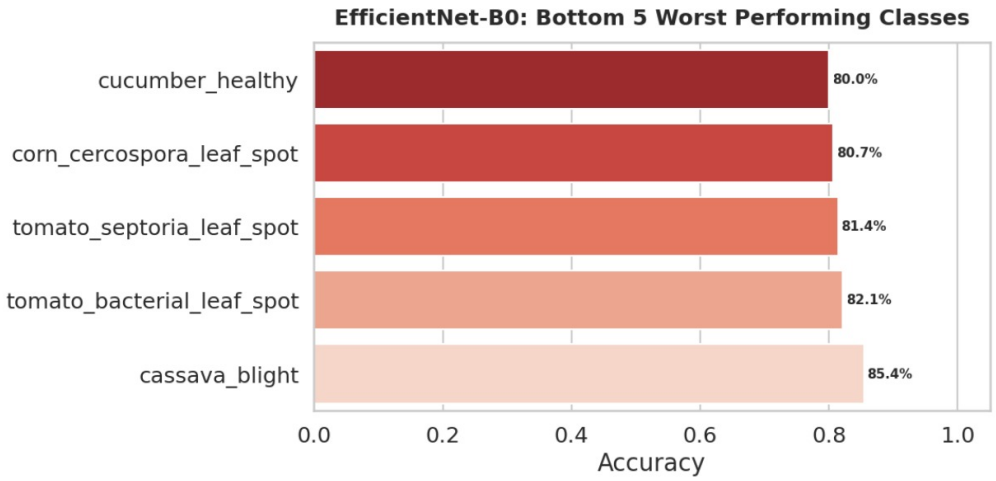

Figure 20: Lowest Performing Disease Classes

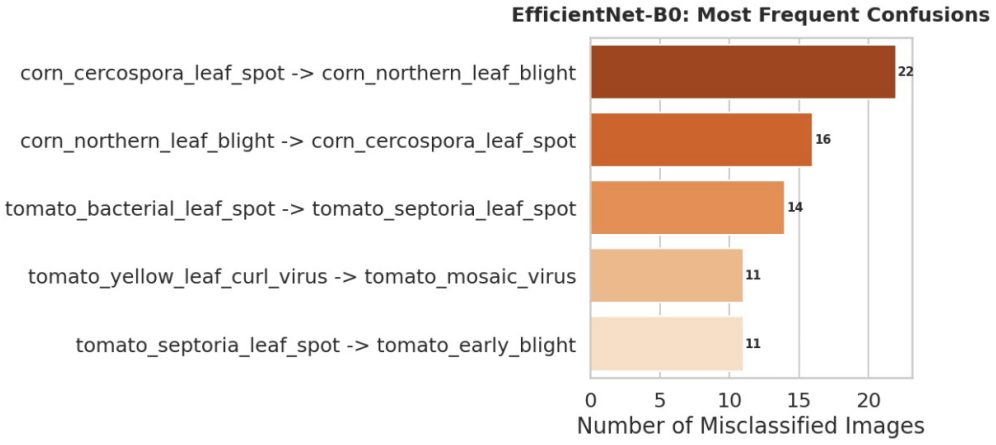

Figure 21: Most Frequent Disease Classification Confusions

4.3.2 VNIR Model Performance Metrics (PSNR and SSIM)

The Thanal subsystem was developed to estimate virtual near-infrared (VNIR) imagery directly from RGB inputs, thereby enabling physiological stress assessment without requiring specialized multispectral imaging equipment. Unlike the disease classification task, which involves categorical prediction, VNIR estimation is an image-to-image translation problem. Consequently, evaluation focused on measuring reconstruction quality between generated VNIR images and corresponding ground-truth near-infrared references.

The VNIR estimation model utilizes an Attention U-Net architecture in which attention gates selectively emphasize relevant plant regions during feature reconstruction while suppressing background information. Prior to inference, an HSV-based preprocessing pipeline isolates leaf tissue from complex environmental backgrounds, reducing noise and improving reconstruction quality. This combination of classical computer vision techniques and deep learning-based image synthesis forms the foundation of the Thanal monitoring pipeline.

Performance evaluation was conducted using two widely accepted image reconstruction metrics: Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM). These metrics were selected because they provide complementary assessments of image quality.

PSNR evaluates pixel-level reconstruction fidelity by measuring the difference between predicted VNIR images and their corresponding ground-truth targets. Higher PSNR values indicate lower reconstruction error and greater numerical similarity between generated and reference images.

SSIM evaluates perceptual similarity by considering structural information, luminance consistency, and contrast preservation. Since plant stress monitoring depends heavily on preserving leaf structure and reflectance patterns, SSIM provides a particularly meaningful assessment of the practical usefulness of the generated VNIR representations.

Table 8: VNIR Reconstruction Performance Metrics

Metric	**Value**
PSNR	29.8 dB
SSIM	0.89

The achieved PSNR value indicates that the generated VNIR images closely approximate the true near-infrared reflectance distributions with relatively low reconstruction noise. From a practical perspective, this level of reconstruction fidelity suggests that the model is capable of producing VNIR estimates that preserve the radiometric information required for subsequent stress monitoring calculations.

Similarly, the SSIM score of 0.89 demonstrates strong structural agreement between generated and reference images. The model successfully preserves important leaf features, including boundaries, venation structures, and spatial reflectance variations. This observation is particu-

larly significant because the stress detection logic relies on extracting VNIR-derived statistics from localized leaf regions rather than merely generating visually plausible images.

Qualitative inspection of generated VNIR outputs further supports the quantitative findings. The reconstructed images exhibit clear correspondence with the underlying leaf structures present in the reference NIR imagery, indicating that the attention-gated architecture successfully learns meaningful spectral relationships between RGB and near-infrared domains.

The combination of a PSNR value of 29.8 dB and an SSIM score of 0.89 demonstrates that the Thanal subsystem achieves sufficiently accurate VNIR reconstruction for integration into the NAVA stress monitoring workflow. These results validate the feasibility of utilizing software-generated VNIR representations as a low-cost alternative to dedicated multispectral sensing hardware for early stress detection applications.

4.4 Qualitative Analysis

4.4.1 Chat Context and Memory Evaluation

The conversational intelligence capabilities of NAVA were evaluated through a series of structured interaction scenarios designed to assess contextual reasoning, memory persistence, note extraction, and farm-aware response generation. Unlike conventional chatbot evaluations that focus primarily on linguistic quality, the objective of these experiments was to determine whether Mozhi could maintain meaningful agricultural context across multiple conversational turns and extended interactions.

The evaluation demonstrated that the system successfully incorporated farm-specific contextual information during dialogue generation. When queried about farm conditions, the assistant was able to retrieve and summarize field-level information including crop inventories, growth stages, disease records, and monitoring results stored within the platform. Responses were generated using contextual information assembled from the Shared Foundation Layer rather than relying solely on language model priors.

The experiments also validated the hierarchical memory architecture implemented within Mozhi. Conversation histories were automatically condensed into Level-1 summaries once the active message threshold was exceeded. As interactions accumulated, these summaries were further consolidated into Level-2 long-term memory rollups. This process preserved important conversational information while preventing excessive context growth. The evaluation confirmed that relevant historical information remained available during later interactions despite the original messages no longer being present in the active context window.

An additional aspect of the evaluation involved automated agronomic note extraction. Following conversations involving disease treatments and farm management actions, the system successfully identified actionable agronomic events and converted them into structured crop

notes. These notes subsequently became available as contextual information during future interactions, thereby establishing continuity between conversational history and farm record management.

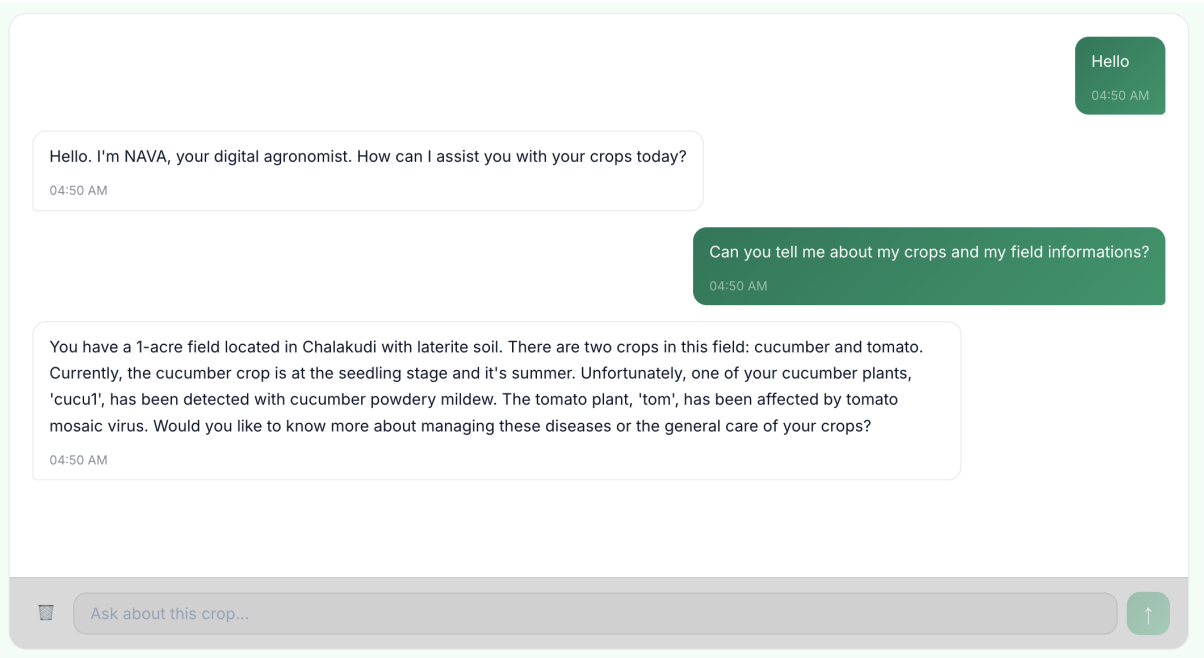

Figure 22: Example Conversation Demonstrating Farm Context Injection

The results indicate that Mozhi functions not merely as a conversational interface but as a persistent farm-aware assistant capable of maintaining long-term contextual awareness throughout a cultivation cycle.

4.4.2 RAG System Evaluation (With vs. Without RAG)

The effectiveness of the Yukthi retrieval pipeline was evaluated through comparative analysis of responses generated with retrieval augmentation enabled and disabled. The purpose of this experiment was to examine the extent to which external agricultural knowledge improved response quality, factual grounding, and domain specificity.

The evaluation used agricultural advisory queries related to disease diagnosis and treatment recommendations. When retrieval augmentation was enabled, the system first identified relevant knowledge sources using its hybrid retrieval strategy and subsequently incorporated retrieved document chunks into the prompt supplied to the response-generation model. The retrieved content included document names, section references, and agronomically relevant passages extracted from authoritative agricultural resources.

The responses generated with retrieval augmentation exhibited substantially greater specificity and traceability. Recommendations included detailed disease management strategies supported by information retrieved from agricultural extension documents. Furthermore, the user interface

exposed the retrieved sources and supporting passages, allowing users to inspect the evidence underlying generated recommendations.

In contrast, responses generated without retrieval augmentation relied entirely on the language model's internal knowledge. Although these responses remained generally relevant, they lacked explicit source grounding and frequently provided broader, less specific recommendations. The absence of verifiable references reduced transparency and made it more difficult to validate the correctness of the generated advice.

4. Cucumber — Powdery Mildew

Question asked:

> My cucumber plants have white powdery patches on the upper surface of the leaves. What is Powdery Mildew in cucumber, what causes it, and what are the treatment and prevention measures?

	With RAG	Without RAG
RAG active	✅ Yes	❌ No
Chunks retrieved	3	0

✅ Answer WITH RAG

> Powdery mildew is a common fungal disease affecting cucumbers, characterized by white, powdery patches on the upper surface of leaves. The causal fungi are Podosphaera xanthii and Erysiphe cichoracearum.
>
> The disease develops quickly under favorable conditions, which include dense plant growth, low light intensity, high relative humidity, and temperatures between 68-80°F (20-27°C).
>
> Symptoms include white, powdery fungal growth on the upper surface of leaves, stems, and fruits. As the disease progresses, infected leaves become yellow, distorted, and may drop prematurely.
>
> To manage powdery mildew, a combination of cultural, biological, and chemical controls is recommended. Here are some measures:
>
> **Cultural Controls:**
>
> 1. **Plant resistant varieties**: Many commercial cucumber cultivars have built-in resistance to powdery mildew.
> 2. **Improve air circulation**: Increase row and plant spacing to reduce humidity and promote air movement.
> 3. **Water carefully**: Avoid overhead watering, and instead use drip irrigation or soaker hoses to reduce leaf wetness.
> 4. **Remove infected leaves**: Dispose of infected leaves to prevent the spread of the disease.
>
> **Biological Controls:**
>
> 1. **Fungicides containing antagonistic fungi**: Apply biological control products containing fungi that antagonize the powdery mildew pathogen.
>
> **Chemical Controls:**
>
> 1. **Fungicides**: Apply fungicides containing mobile or systemic ingredients, such as Vivando, Proline, or Procure, every 7-10 days starting at the onset of disease development. Always follow the recommended label rates and resistance management strategies to minimize the risk of fungicide resistance.

❌ **Answer WITHOUT RAG**

> Powdery Mildew is a common fungal disease affecting cucumbers, especially in humid and warm conditions like those in Kottayam, Kerala during the Monsoon Season. The white powdery patches on the upper surface of the leaves are a clear indication of this disease.
>
> Causes: Powdery Mildew is caused by the fungus Podosphaera xanthii. It thrives in humid environments with temperatures between 15°C to 25°C, which is exactly the current condition in your field. High humidity, overhead watering, and poor air circulation can contribute to the development of this disease.
>
> Treatment: For mild infections, remove and dispose of infected leaves to prevent the disease from spreading. For more severe cases, consider using fungicides containing copper, sulfur, or neem oil. However, please consult local guidelines or an agronomist for specific dosage recommendations.
>
> Prevention Measures: 1. Improve air circulation: Ensure adequate spacing between plants and rows to promote good air movement. 2. Water management: Avoid overhead watering, and instead, use drip irrigation or soaker hoses to reduce moisture on the leaves. 3. Crop monitoring: Regularly inspect your plants for signs of Powdery Mildew and take action promptly. 4. Resistant varieties: Consider growing cucumber varieties with built-in resistance to Powdery Mildew. 5. Sanitation: Remove weeds and debris from the field, as they can harbor the fungus.
>
> Remember to prioritize disease detection results and take necessary actions to prevent the spread of Powdery Mildew in your cucumber crop.

Figure 23: Responses With vs. Without RAG

The evaluation therefore demonstrates that retrieval augmentation significantly improves the reliability, specificity, and verifiability of agricultural recommendations. By grounding responses in curated agricultural knowledge sources, Yukthi reduces the risk of unsupported recommendations and enhances user trust in the advisory process.

4.4.3 Grad-CAM Heatmap Analysis

Explainability constitutes an important requirement for AI-assisted agricultural diagnosis, particularly when recommendations may influence crop management decisions. To address this requirement, the disease classification pipeline incorporates Grad-CAM visualization, enabling inspection of the image regions that contribute most strongly to classification decisions.

Qualitative analysis across multiple crop species demonstrated that the generated heatmaps consistently concentrated on symptomatic leaf regions rather than irrelevant background features. In diseased samples, activation maps were typically localized around lesions, discoloration patterns, necrotic tissue, and disease-specific visual abnormalities. This behavior suggests that the classifier learns agriculturally meaningful features rather than exploiting spurious correlations within the dataset.

The analysis additionally validated the reliability gating mechanism implemented within Mizhi. Predictions with confidence values below the predefined reliability threshold were marked as unreliable and did not produce Grad-CAM visualizations. This behavior prevents potentially misleading explanations from being presented when model confidence is insufficient to support

dependable diagnosis.

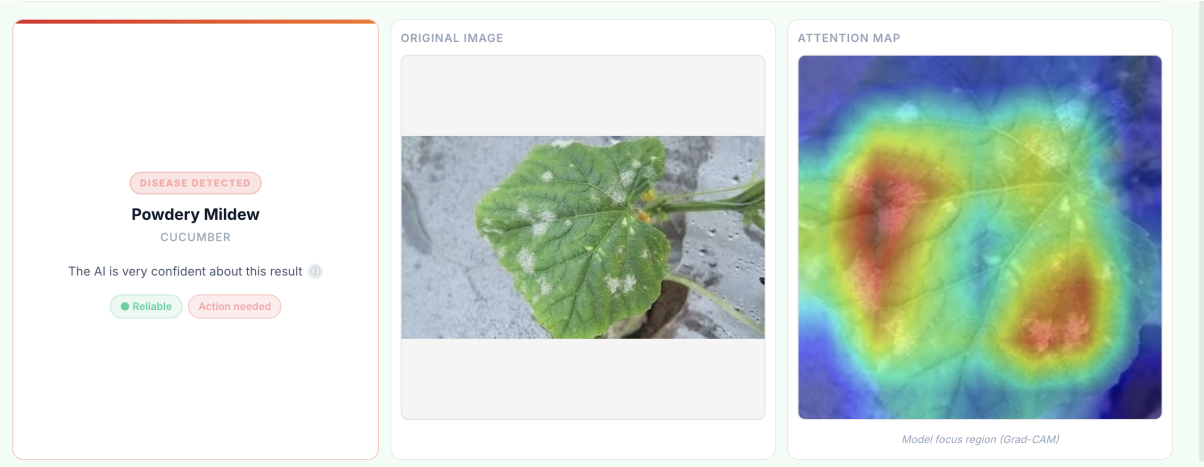

Figure 24: Grad-CAM Visualizations of Disease Regions Across Crops

Across the evaluated crop categories, the visual correspondence between highlighted regions and observable disease symptoms provided evidence that the model's decision-making process aligns with agronomically relevant characteristics. The explainability layer therefore contributes both interpretability and operational safety to the disease diagnosis workflow.

4.4.4 VNIR Monitoring Analysis

The qualitative evaluation of the Thanal subsystem focused on assessing the effectiveness of the VNIR monitoring workflow for proactive stress detection. Unlike traditional disease diagnosis systems that operate after visible symptoms appear, Thanal attempts to identify physiological changes through estimated near-infrared reflectance patterns.

The monitoring evaluation was conducted in two phases. The first phase involved baseline establishment using healthy plant observations. During this calibration stage, the system accumulated VNIR ratio measurements and computed a reference baseline representing the expected physiological condition of the monitored plant. Once sufficient observations were collected, baseline generation was completed and the plant transitioned into active monitoring mode.

The second phase evaluated stress detection behavior using diseased plant imagery. The system successfully identified significant deviations from baseline reflectance characteristics and generated warning or critical alerts when predefined thresholds were exceeded. These alerts were derived from changes in the VNIR-to-green reflectance ratio rather than visible disease symptoms, demonstrating the intended proactive monitoring capability.

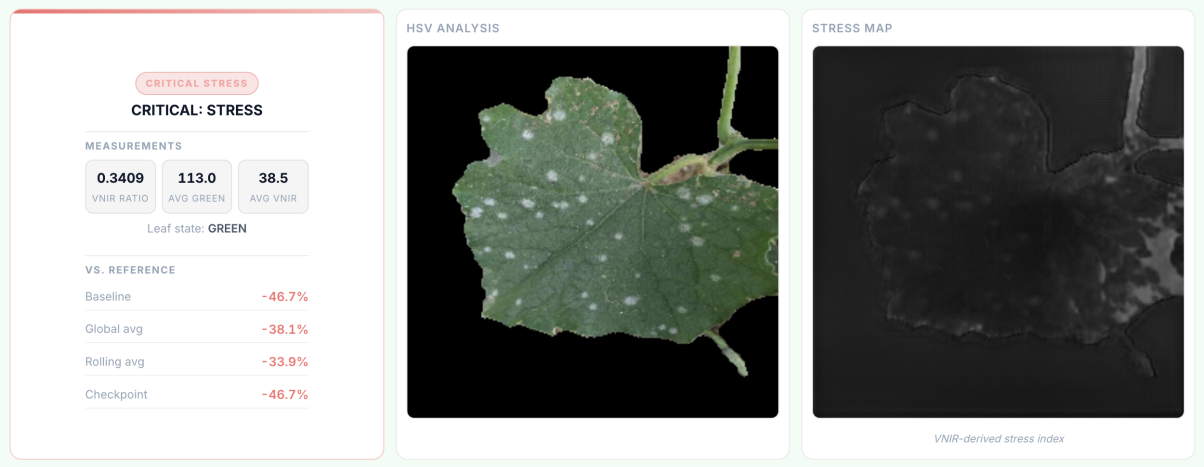

Figure 25: VNIR Stress Detection Event

Qualitative inspection of the generated VNIR maps further revealed clear differences between healthy and stressed plant samples. Healthy observations generally produced brighter reflectance patterns, whereas stressed samples exhibited darker regions corresponding to reduced estimated near-infrared reflectance. These visual differences aligned with the quantitative monitoring statistics reported by the system.

The evaluation indicates that the VNIR monitoring framework successfully establishes individualized plant baselines and detects subsequent physiological deviations. This capability extends the functionality of NAVA beyond conventional disease diagnosis by providing an additional mechanism for early stress assessment and continuous plant health monitoring.

4.5 Summary of Findings

The experimental evaluation demonstrates that the NAVA platform successfully integrates computer vision, virtual near-infrared estimation, retrieval-augmented generation, conversational intelligence, contextual memory management, and farm-level contextual reasoning within a unified agricultural intelligence framework.

From a quantitative perspective, EfficientNet-B0 emerged as the most effective disease classification architecture among the evaluated candidates, achieving a test accuracy of 91.44% together with balanced precision, recall, and F1-score values of 92.00%. The Thanal VNIR estimation model achieved a PSNR of 29.8 dB and an SSIM score of 0.89, indicating strong reconstruction fidelity and structural preservation.

Qualitative evaluations further demonstrated the effectiveness of the higher-level intelligence components. The hierarchical memory architecture successfully preserved long-term conversational context, retrieval augmentation improved the specificity and verifiability of agricultural recommendations, Grad-CAM visualizations provided interpretable diagnostic explanations,

and the VNIR monitoring framework enabled proactive stress assessment through baseline-based physiological analysis.

Taken together, these findings indicate that the individual modules operate effectively both independently and as part of the integrated NAVA ecosystem. The results validate the feasibility of combining perception, retrieval, reasoning, memory, and monitoring capabilities within a single digital agronomist platform capable of supporting informed agricultural decision-making.

5 Conclusion and Future Scope

This project presented the design and implementation of **NAVA (Next-Gen Agricultural Virtual Assistant)**, a multi-modal agricultural intelligence platform that integrates computer vision, virtual near-infrared estimation, retrieval-augmented generation, conversational artificial intelligence, contextual memory management, weather integration, and farm management capabilities within a unified architecture. The system was developed as five interconnected modules—Gathi, Mizhi, Mozhi, Yukthi, and the Shared Foundation Layer—each addressing a distinct aspect of agricultural assistance while operating together as a cohesive platform. The modular architecture facilitated the integration of perception, reasoning, knowledge retrieval, and contextual data management, enabling the system to support multiple agricultural workflows within a single application.

5.1 Summary

The experimental evaluation demonstrated that the objectives of the project were achieved within the intended scope. Comparative evaluation of MobileNetV2, ResNet-50, and EfficientNet-B0 identified EfficientNet-B0 as the most suitable disease classification model, achieving an overall classification accuracy of **91.44%**, with a precision, recall, and F1-score of **0.92**. The Thanal VNIR estimation model achieved a **PSNR of 29.8 dB** and an **SSIM of 0.85**, demonstrating satisfactory reconstruction quality for stress monitoring. The integrated evaluation further verified the successful operation of contextual memory management, retrieval-augmented generation, Grad-CAM explainability, and rolling-baseline stress monitoring within the complete platform. Overall, the work demonstrates the feasibility of integrating multiple artificial intelligence techniques into a unified agricultural assistance system that supports crop monitoring, knowledge retrieval, and context-aware conversational interaction.

5.2 Limitations of the Present Work

Although the proposed system achieved the objectives defined for this project, several limitations remain that present opportunities for future improvement.

The disease diagnosis module currently supports seven crop species and twenty-seven disease classes, limiting its applicability to a broader range of crops and plant diseases encountered in practical agricultural environments. Expanding both crop coverage and disease diversity would improve the versatility of the platform. The VNIR estimation framework was developed using a relatively limited training dataset. While the model demonstrated promising reconstruction performance, the availability of larger and more diverse datasets would likely improve its ability to generalize across different crop species, environmental conditions, and imaging scenarios.

The retrieval-augmented generation pipeline currently operates on a relatively small agricultural

knowledge base. As a result, only a limited number of highly relevant document chunks are available for many queries, which may occasionally lead to the retrieval of partially relevant contextual information. Increasing both the size and diversity of the knowledge base would improve retrieval quality and strengthen the factual grounding of generated responses.

5.3 Future Work

Several directions can be explored to further enhance the capabilities of the NAVA platform.

One important extension would be the development of a native mobile application, enabling better integration with smartphone hardware, offline functionality, and improved usability for farmers in field conditions. Future work should also focus on improving the existing machine learning models by collecting larger and more diverse datasets. An extended comparative study involving additional deep learning architectures for both disease classification and VNIR estimation could then be conducted to identify models that provide improved performance under expanded datasets.

The retrieval-augmented generation framework can be further strengthened by incorporating larger agricultural knowledge bases together with more advanced retrieval strategies, reranking techniques, and retrieval optimization methods to improve the relevance of retrieved information. Improving accessibility is another important direction. Introducing multilingual conversational support, particularly for regional languages, together with voice-based interaction would allow the platform to be used more effectively by a wider range of users.

Another useful enhancement would be the automatic generation of farm health reports that consolidate disease diagnoses, stress monitoring history, weather information, and farm activities. Such reports could support farm record keeping and may also be useful for applications such as crop insurance documentation and other agricultural administrative processes. Finally, future versions of the platform could incorporate role-based access control, allowing different categories of users—such as farm owners, field workers, agricultural officers, and consultants—to access information and functionalities appropriate to their respective responsibilities. This would improve collaboration while maintaining appropriate separation of user privileges.

In summary, this project demonstrates an approach to integrating multiple artificial intelligence techniques within a single agricultural assistance platform. By combining computer vision, spectral estimation, retrieval-augmented generation, conversational intelligence, and contextual information management, NAVA provides a unified framework for supporting various aspects of crop monitoring and agricultural decision-making. Although the system can be further improved through expanded datasets, enhanced retrieval methods, and broader deployment capabilities, the work presented in this dissertation establishes a foundation for continued research and development in AI-assisted agriculture.

References

[1] S. Islam, "Agriculture, food security, and sustainability: A review," *Exploration of Foods and Foodomics*, Apr. 2025, doi: 10.37349/eff.2025.101082.

[2] A. K. Basantaray, "Research landscape of marginal and small farmers in India from 1975 to 2024: A bibliometric analysis," *Journal of Tropical Agriculture*, vol. 63, no. 1&2, pp. 21–28, Jun. 2025, doi: 10.63599/jta.2025.1518.

[3] M. Carvajal-Yepes *et al.*, "A global surveillance system for crop diseases," *Science*, vol. 364, no. 6447, pp. 1237–1239, Jun. 2019, doi: 10.1126/science.aaw1572.

[4] P. N. Thotad, S. Kallur, and A. Nandeppanavar, "An Efficient Model for Plant Disease Detection in Agriculture Using Deep Learning Approaches," in *2023 4th IEEE Global Conference for Advancement in Technology (GCAT)*, pp. 1–6, Oct. 2023, doi: 10.1109/GCAT59970.2023.10353343.

[5] S. O. Araújo, R. S. Peres, J. C. Ramalho, F. Lidon, and J. Barata, "Machine learning applications in agriculture: Current trends, challenges, and future perspectives," *Agronomy*, vol. 13, no. 12, p. 2976, Dec. 2023, doi: 10.3390/agronomy13122976.

[6] A. Jafar, N. Bibi, R. A. Naqvi, A. Sadeghi-Niaraki, and D. Jeong, "Revolutionizing agriculture with artificial intelligence: Plant disease detection methods, applications, and their limitations," *Frontiers in Plant Science*, vol. 15, p. 1356260, Mar. 2024, doi: 10.3389/fpls.2024.1356260.

[7] A. Ahmad *et al.*, "AI can empower agriculture for global food security: Challenges and prospects in developing nations," *Frontiers in Artificial Intelligence*, vol. 7, p. 1328530, Apr. 2024, doi: 10.3389/frai.2024.1328530.

[8] M. Fanuel *et al.*, "AgriRegion: Region-Aware Retrieval for High-Fidelity Agricultural Advice," *arXiv preprint*, Dec. 2025, doi: 10.48550/arXiv.2512.10114.

[9] Z. Zhao, B. Jia, H. Gao, Y. Han, and Y. Zhou, "A Multi-Modal Fusion Framework with Uncertainty Estimation for Plant Disease Detection Using RGB and Hyperspectral Imaging," in *2025 4th International Conference on Image Processing, Computer Vision and Machine Learning (ICICML)*, pp. 499–505, Nov. 2025, doi: 10.1109/ICICML67980.2025.11333809.

[10] C. Yan *et al.*, "CDIP-ChatGLM3: A dual-model approach integrating computer vision and language modeling for crop disease identification and prescription," *Computers and Electronics in Agriculture*, vol. 236, p. 110442, Apr. 2025, doi: 10.1016/j.compag.2025.110442.

[11] A. Picon *et al.*, "Deep convolutional neural network for damaged vegetation segmentation from RGB images based on virtual NIR-channel estimation," *Artificial Intelligence in Agriculture*, vol. 6, pp. 199–210, Jan. 2022, doi: 10.1016/j.aiia.2022.09.004.

[12] D. K, B. S, L. K, and O. J, "HVAF-XAI-Net: A multimodal explainable AI framework for accurate mulberry leaf disease detection," *Smart Agricultural Technology*, vol. 12, p. 101377, Aug. 2025, doi: 10.1016/j.atech.2025.101377.

[13] M. F. Guerri, C. Distante, P. Spagnolo, F. Bougourzi, and A. Taleb-Ahmed, "Deep learning techniques for hyperspectral image analysis in agriculture: A review," *ISPRS Open Journal of Photogrammetry and Remote Sensing*, vol. 12, p. 100062, Mar. 2024, doi: 10.1016/j.ophoto.2024.100062.

[14] A. Chourlias, J. Violos, and A. Leivadeas, "Virtual sensors for smart farming: An IoT- and AI-enabled approach," *Internet of Things*, vol. 32, p. 101611, May 2025, doi: 10.1016/j.iot.2025.101611.

[15] X. Chen *et al.*, "A vision-language foundation model-based multi-modal retrieval-augmented generation framework for remote sensing lithological recognition," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 225, pp. 328–340, May 2025, doi: 10.1016/j.isprsjprs.2025.04.015.

[16] J. Jiang *et al.*, "Knowledge assimilation: Implementing knowledge-guided agricultural large language model," *Knowledge-Based Systems*, vol. 314, p. 113197, Feb. 2025, doi: 10.1016/j.knosys.2025.113197.

[17] R. A. Naagarajan and S. Streif, "Enhancing greenhouse management with interpretable AI: A natural language interface for advanced and optimization-based control," *Smart Agricultural Technology*, vol. 11, p. 101041, May 2025, doi: 10.1016/j.atech.2025.101041.

[18] A. Deo, N. Sawant, A. Arora, and S. Karmakar, "How has scientific literature addressed crop planning at farm level: A bibliometric-qualitative review," *Farming System*, vol. 3, no. 2, p. 100139, Jan. 2025, doi: 10.1016/j.farsys.2025.100139.

[19] Y. Yang *et al.*, "EMWQ: An efficient mixed precision weight quantization method for large language models," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 1–13, Jan. 2025, doi: 10.1109/TNNLS.2025.3585677.

[20] Y. Wu, R. Lin, J. Que, Q. Zeng, and H. Zhang, "KVC-Q: A high-fidelity and dynamic KV Cache quantization framework for long-context large language models," *Journal of Systems Architecture*, vol. 173, p. 103699, Jan. 2026, doi: 10.1016/j.sysarc.2026.103699.

[21] I. Attri, L. K. Awasthi, T. P. Sharma, and P. Rathee, "A review of deep learning techniques used in agriculture," *Ecological Informatics*, vol. 77, p. 102217, Jul. 2023, doi: 10.1016/j.ecoinf.2023.102217.

[22] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint*, Nov. 2015, doi: 10.48550/arXiv.1511.08060.

[23] T. Wei, Z. Chen, Z. Huang, and X. Yu, "Benchmarking in-the-wild multimodal disease recognition and a versatile baseline," *arXiv preprint*, Aug. 2024, doi: 10.48550/arXiv.2408.03120.

[24] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," *arXiv preprint*, Nov. 2019, doi: 10.48550/arXiv.1911.10317.

[25] Petchiammal, B. Kiruba, Murugan, and P. Arjunan, "Paddy Doctor: A visual image dataset for automated paddy disease classification and benchmarking," in *Proceedings of the 6th Joint International Conference on Data Science & Management of Data (CODS-COMAD '23)*, pp. 203–207, Jan. 2023, doi: 10.1145/3570991.3570994.

[26] N. Bevers, E. J. Sikora, and N. B. Hardy, "Pictures of diseased soybean leaves by category captured in field and with controlled backgrounds: Auburn Soybean Disease Image Dataset (ASDID)," *Open MIND*, Oct. 27, 2022, doi: 10.5061/dryad.41ns1rnj3.

[27] I. S. J. Y. L. H. S. A. B. MacDonald, "DeepNIR: Datasets for generating synthetic NIR images and improved fruit detection system using deep learning techniques," *Sensors*, vol. 22, no. 13, p. 4721, Jun. 2022, doi: 10.3390/s22134721.

[28] FastAPI, "FastAPI," [Online]. Available: https://fastapi.tiangolo.com/. Accessed: Jul. 2, 2026.

[29] React Team, "React v18.0," Mar. 29, 2022. [Online]. Available: https://react.dev/blog/2022/03/29/react-v18. Accessed: Jul. 2, 2026.

[30] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," *arXiv preprint*, May 2019, doi: 10.48550/arXiv.1905.11946.

[31] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *arXiv preprint*, Oct. 2016, doi: 10.48550/arXiv.1610.02391.

[32] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, pp. 234–241, May 2015, doi: 10.48550/arXiv.1505.04597.

Appendix

Disease Detection

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Optional, Tuple

import torch
from PIL import Image
from torchvision import models, transforms

from nava_core.shared.utils.logging import get_logger
from nava_core.shared.utils.paths import models_dir
from .gradcam import GradCamGenerator
from .labels import load_labels

log = get_logger("mizhi.detection")


@dataclass
class PredictionResult:
    class_index: int
    class_label: str
    confidence: float
    reliability: str


def _default_model_path() -> Path:
    return models_dir() / "EfficientNet-B0.pth"


def _default_labels_path() -> Path:
    return models_dir() / "EfficientNet-B0-labels.txt"


def _build_model(num_classes: int) -> torch.nn.Module:
    model = models.efficientnet_b0(weights=None)
    model.classifier[1] = torch.nn.Linear(
```

```
        model.classifier[1].in_features, num_classes
    )
    return model


def _extract_state_dict(checkpoint: object) ->
↪   Tuple[Optional[torch.nn.Module], Optional[dict]]:
    if isinstance(checkpoint, torch.nn.Module):
        return checkpoint, None
    if isinstance(checkpoint, dict):
        for key in ("state_dict", "model_state_dict", "model"):
            if key in checkpoint:
                return None, checkpoint[key]
        return None, checkpoint
    return None, None


def _clean_state_dict(state_dict: dict) -> dict:
    return {
        (k[7:] if k.startswith("module.") else k): v
        for k, v in state_dict.items()
    }


class EfficientNetB0Predictor:
    def __init__(
        self,
        model_path: Optional[Path] = None,
        labels_path: Optional[Path] = None,
        device: str = "cpu",
        confidence_threshold: float = 0.80,
    ) -> None:
        self.device = torch.device(device)
        self.model_path = model_path or _default_model_path()
        self.labels_path = labels_path or _default_labels_path()
        self.labels = load_labels(self.labels_path)
        self.confidence_threshold = confidence_threshold

        self.model = _build_model(num_classes=len(self.labels))
        checkpoint = torch.load(self.model_path, map_location="cpu")
        model_obj, state_dict = _extract_state_dict(checkpoint)

```

```
        if model_obj is not None:
            self.model = model_obj
        elif state_dict is not None:
            self.model.load_state_dict(_clean_state_dict(state_dict),
            ↪  strict=True)
        else:
            raise ValueError("Unsupported checkpoint format")

        self.model.to(self.device)
        self.model.eval()

        self._transform = transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
            ↪  0.224, 0.225]),
        ])

        self._cam = GradCamGenerator(self.model, self.model.features[-1])
        log.info("EfficientNet-B0 loaded: %d classes, device=%s",
        ↪  len(self.labels), device)

    def _preprocess(self, image: Image.Image) -> Tuple[torch.Tensor,
    ↪  Image.Image]:
        image = image.convert("RGB")
        # Keep the cropped version for Grad-CAM overlay
        cropped = transforms.CenterCrop(224)(transforms.Resize(256)(image))
        tensor = self._transform(image)
        return tensor, cropped

    def predict(self, image: Image.Image) -> PredictionResult:
        #Run inference only | no Grad-CAM.
        tensor, _ = self._preprocess(image)
        input_tensor = tensor.unsqueeze(0).to(self.device)
        with torch.no_grad():
            logits = self.model(input_tensor)
            probs = torch.softmax(logits, dim=1)
            confidence, class_index = torch.max(probs, dim=1)

        idx = int(class_index.item())
        conf = float(confidence.item())
```

```
        label = self.labels[idx] if idx < len(self.labels) else "unknown"
        reliability = "RELIABLE" if conf >= self.confidence_threshold else
        ↪  "UNRELIABLE"
        return PredictionResult(class_index=idx, class_label=label,
        ↪  confidence=conf, reliability=reliability)

    def predict_with_cam(self, image: Image.Image) -> Tuple[PredictionResult,
    ↪  Image.Image]:

        #Single forward pass for prediction + Grad-CAM overlay.
        #NOTE: No torch.no_grad() wrapper | Grad-CAM requires gradient flow.

        tensor, cropped = self._preprocess(image)
        input_tensor = tensor.unsqueeze(0).to(self.device)

        # Forward pass (Grad-CAM will compute gradients internally)
        logits = self.model(input_tensor)
        probs = torch.softmax(logits, dim=1)
        confidence, class_index = torch.max(probs, dim=1)

        idx = int(class_index.item())
        conf = float(confidence.item())
        label = self.labels[idx] if idx < len(self.labels) else "unknown"
        reliability = "RELIABLE" if conf >= self.confidence_threshold else
        ↪  "UNRELIABLE"

        cam_image = self._cam.generate(input_tensor, cropped, idx)

        result = PredictionResult(class_index=idx, class_label=label,
        ↪  confidence=conf, reliability=reliability)
        log.info("Prediction: %s (%.2f%%) - %s", label, conf * 100,
        ↪  reliability)
        return result, cam_image
```

NIR Monitoring

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
```

```python
from typing import Tuple

import cv2
import numpy as np
from PIL import Image

from nava_core.shared.utils.logging import get_logger
from .analyzer import VNIRAnalyzer, VNIRStats
from .inference import VNIREngine
from .validation import validate_plant_id

log = get_logger("mizhi.vnir.pipeline")


@dataclass
class LeafIsolationResult:
    leaf_state: str
    leaf_mask: np.ndarray
    hsv_visual: np.ndarray
    masked_rgb: np.ndarray


class VNIRPipeline:
    def __init__(
        self,
        model_path: Path | None = None,
        stress_threshold_pct: float = 15.0,
        warning_threshold_pct: float = 10.0,
    ) -> None:
        self.engine = VNIREngine(model_path=model_path)
        self.analyzer = VNIRAnalyzer(
            stress_threshold_pct=stress_threshold_pct,
            warning_threshold_pct=warning_threshold_pct,
        )

    def isolate_leaf(self, frame_bgr: np.ndarray) -> LeafIsolationResult:
        frame_256 = cv2.resize(frame_bgr, (256, 256))
        hsv_frame = cv2.cvtColor(frame_256, cv2.COLOR_BGR2HSV)
        total_pixels = 256 * 256

        kernel_large = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))
        kernel_small = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
47
48          # Green leaf mask
49          lower_green = np.array([30, 40, 40])
50          upper_green = np.array([90, 255, 255])
51          green_hsv_mask = cv2.inRange(hsv_frame, lower_green, upper_green)
52          green_hsv_mask = cv2.morphologyEx(green_hsv_mask, cv2.MORPH_CLOSE,
            ↪  kernel_large)
53          green_hsv_mask = cv2.morphologyEx(green_hsv_mask, cv2.MORPH_OPEN,
            ↪  kernel_small)
54
55          green_contours, _ = cv2.findContours(green_hsv_mask,
            ↪  cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
56          max_green_area = 0.0
57          best_green_cnt = None
58          if green_contours:
59              best_green_cnt = max(green_contours, key=cv2.contourArea)
60              max_green_area = cv2.contourArea(best_green_cnt)
61
62          # Yellow-brown stress mask
63          lower_yellow = np.array([15, 70, 70])
64          upper_yellow = np.array([30, 255, 255])
65          yellow_hsv_mask = cv2.inRange(hsv_frame, lower_yellow, upper_yellow)
66          yellow_hsv_mask = cv2.morphologyEx(yellow_hsv_mask, cv2.MORPH_CLOSE,
            ↪  kernel_large)
67          yellow_hsv_mask = cv2.morphologyEx(yellow_hsv_mask, cv2.MORPH_OPEN,
            ↪  kernel_small)
68
69          yellow_contours, _ = cv2.findContours(yellow_hsv_mask,
            ↪  cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
70          max_yellow_area = 0.0
71          best_yellow_cnt = None
72          if yellow_contours:
73              best_yellow_cnt = max(yellow_contours, key=cv2.contourArea)
74              max_yellow_area = cv2.contourArea(best_yellow_cnt)
75
76          leaf_state = "NONE"
77          leaf_mask = np.zeros((256, 256), dtype=np.uint8)
78          contour_bound = np.zeros((256, 256), dtype=np.uint8)
79          min_area = total_pixels * 0.05
80
81          if max_green_area >= max_yellow_area and max_green_area >= min_area:
82              leaf_state = "GREEN"
```

```
            cv2.drawContours(contour_bound, [best_green_cnt], -1, 255, -1)
            leaf_mask = cv2.bitwise_and(green_hsv_mask, contour_bound)
        elif max_yellow_area > max_green_area and max_yellow_area >=
        ↪  min_area:
            leaf_state = "YELLOW_BROWN"
            cv2.drawContours(contour_bound, [best_yellow_cnt], -1, 255, -1)
            leaf_mask = cv2.bitwise_and(yellow_hsv_mask, contour_bound)

        hsv_visual = cv2.cvtColor(hsv_frame, cv2.COLOR_HSV2BGR)
        hsv_visual = cv2.bitwise_and(hsv_visual, hsv_visual, mask=leaf_mask)
        masked_bgr = cv2.bitwise_and(frame_256, frame_256, mask=leaf_mask)
        masked_rgb = cv2.cvtColor(masked_bgr, cv2.COLOR_BGR2RGB)

        return LeafIsolationResult(
            leaf_state=leaf_state,
            leaf_mask=leaf_mask,
            hsv_visual=hsv_visual,
            masked_rgb=masked_rgb,
        )

    def process_image(
        self,
        image: Image.Image,
        plant_id: str,
        history_ratios: list[float],
    ) -> Tuple[VNIRStats, Image.Image, Image.Image]:
        """Run the full pipeline.
        Args:
            image: Input leaf photo.
            plant_id: Validated plant identifier.
            history_ratios: Previous ratio values from the per-user DB.
        Returns:
            (stats, hsv_image, vnir_image)
        """
        frame_bgr = cv2.cvtColor(np.array(image.convert("RGB")),
        ↪  cv2.COLOR_RGB2BGR)
        isolation = self.isolate_leaf(frame_bgr)

        hsv_image = Image.fromarray(cv2.cvtColor(isolation.hsv_visual,
        ↪  cv2.COLOR_BGR2RGB))
        vnir_image = Image.new("L", (256, 256), color=0)

```

```
        if isolation.leaf_state == "GREEN":
            vnir_image =
            ↪  self.engine.predict(Image.fromarray(isolation.masked_rgb))
            vnir_array = np.array(vnir_image).astype(np.float32)
            stats = self.analyzer.analyze(
                isolation.masked_rgb, vnir_array, isolation.leaf_mask,
                ↪  history_ratios
            )
        elif isolation.leaf_state == "YELLOW_BROWN":
            stats = VNIRStats(status="CRITICAL: Visual Stress")
        else:
            stats = VNIRStats(status="No Leaf Detected")

        stats.leaf_state = isolation.leaf_state
        log.info("VNIR scan for %s: %s (state=%s)", plant_id, stats.status,
        ↪  stats.leaf_state)
        return stats, hsv_image, vnir_image

```

Chatbot

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import requests

from nava_core.shared.config import get_settings
from nava_core.shared.utils.logging import get_logger

log = get_logger("mozhi.client")


@dataclass(frozen=True)
class ChatConfig:
    model: str
    url: str
    api_key: str
    timeout: int
```

```python
    temperature: float
    max_new_tokens: int


class ChatClient:
    def __init__(self, config: ChatConfig) -> None:
        self.config = config

    @classmethod
    def from_settings(cls) -> "ChatClient":
        s = get_settings()
        return cls(ChatConfig(
            model=s.hf_model,
            url=s.hf_router_url,
            api_key=s.hf_api_key,
            timeout=s.hf_timeout_seconds,
            temperature=s.hf_temperature,
            max_new_tokens=s.hf_max_new_tokens,
        ))

    def send(
        self,
        messages: list[dict],
        model_override: Optional[str] = None,
        temperature_override: Optional[float] = None,
        max_new_tokens_override: Optional[int] = None,
    ) -> tuple[Optional[str], Optional[str]]:
        if not self.config.api_key:
            return None, "HF_API_KEY not set"

        payload = {
            "model": model_override or self.config.model,
            "messages": messages,
            "temperature": temperature_override if temperature_override is
            ↪  not None else self.config.temperature,
            "max_new_tokens": max_new_tokens_override if
            ↪  max_new_tokens_override is not None else
            ↪  self.config.max_new_tokens,
        }
        headers = {"Authorization": f"Bearer {self.config.api_key}"}

        try:
```

```
            resp = requests.post(self.config.url, headers=headers,
            ↪  json=payload, timeout=self.config.timeout)
            if resp.status_code != 200:
                log.warning("LLM API returned %d: %s", resp.status_code,
                ↪  resp.text[:200])
                return None, f"HTTP {resp.status_code}: {resp.text}"
            choices = resp.json().get("choices", [])
            if not choices:
                return None, "Empty response"
            content = choices[0].get("message", {}).get("content", "")
            return (content or None), (None if content else "Empty response")
        except Exception as exc:
            log.error("LLM request failed: %s", exc)
            return None, f"Network Error: {exc}"
```

RAG

```
"""Query-time RAG retrieval from ChromaDB | LLM-keyword-guided hybrid search.

Pipeline (per query):
  1. [Caller] Router LLM decides RETRIEVE
  2. [Caller] Build enriched query (crop + disease + user message)
  3. [Caller] KeywordExtractor LLM call → 3 precise agronomic keywords
  4. [This module] Semantic search  | top 8 candidates from ChromaDB
  5. [This module] Keyword searches | for each LLM keyword, top 3 from
     ChromaDB where_document filter → ~9 keyword candidates (deduped to 8)
  6. Merge + deduplicate all candidates (up to 16 unique chunks)
  7. Rerank: combined score = 0.7 × semantic_similarity + 0.3 ×
  ↪  keyword_overlap
  8. Return top 4

Why 8+8 → top 4:
  - 8 semantic ensures broad topical coverage
  - LLM-derived keywords (not heuristic stopword filtering) target specific
    disease names and agronomic terms, pulling in chunks that narrowly miss
    the semantic top-8 (e.g. two different Sigatoka disease entries)
  - Reranking to 4 (up from 3) gives the LLM richer reference material
    without overwhelming the context window
```

```
"""

from __future__ import annotations

import re
from dataclasses import dataclass
from typing import Optional

from .store import YukthiStore
from nava_core.shared.utils.logging import get_logger

log = get_logger("yukthi.retriever")

# Fallback heuristic stop-words used when LLM keyword extraction fails
_STOPWORDS = frozenset({
    "a", "an", "the", "is", "are", "was", "were", "be", "been", "being",
    "have", "has", "had", "do", "does", "did", "will", "would", "could",
    "should", "may", "might", "can", "my", "your", "our", "its", "this",
    "that", "these", "those", "what", "how", "why", "when", "where",
    "which", "who", "and", "or", "but", "in", "on", "at", "to", "for",
    "of", "with", "by", "from", "up", "about", "into", "through",
    "i", "me", "we", "you", "it", "he", "she", "they", "crop", "banana",
    "tell", "give", "show", "want", "know", "please", "need", "help",
})


@dataclass
class RAGChunk:
    text: str
    source: str
    section: str
    score: float   # cosine distance | lower is more similar (0.0 =
    ↪ identical)


class RAGRetriever:
    """Hybrid semantic + LLM-keyword retriever backed by ChromaDB +
    ↪ sentence-transformers."""

    def __init__(
        self,
        store: YukthiStore,
```

```
        embed_model: str = "BAAI/bge-small-en-v1.5",
        top_k: int = 4,
        distance_threshold: float = 0.45,
    ) -> None:
        self.store = store
        self.embed_model = embed_model
        self.top_k = top_k
        self.distance_threshold = distance_threshold
        self._encoder = None  # singleton, lazy-loaded

    @classmethod
    def from_settings(cls, store: Optional[YukthiStore] = None) ->
    ↪  "RAGRetriever":
        from nava_core.shared.config import get_settings
        s = get_settings()
        return cls(
            store=store or YukthiStore(s.yukthi_chroma_dir),
            embed_model=s.yukthi_embed_model,
            top_k=s.yukthi_top_k,
            distance_threshold=s.yukthi_distance_threshold,
        )

    def _get_encoder(self):
        if self._encoder is None:
            try:
                from sentence_transformers import SentenceTransformer
                log.info("Loading retrieval embedding model: %s",
                ↪  self.embed_model)
                self._encoder = SentenceTransformer(self.embed_model)
                log.info("Embedding model ready.")
            except ImportError:
                raise RuntimeError(
                    "sentence-transformers is required. Install it with: pip
                    ↪  install sentence-transformers"
                )
        return self._encoder

    #  Keyword scoring

    def _keyword_score(self, document: str, search_terms: list[str]) ->
    ↪  float:
```

```
98          """Keyword overlap ratio [0.0, 1.0] | how many search terms appear in
            ↪ document."""
99          if not search_terms:
100             return 0.0
101         doc_lower = document.lower()
102         hits = sum(1 for t in search_terms if t.lower() in doc_lower)
103         return hits / len(search_terms)
104
105     def _heuristic_fallback_terms(self, text: str) -> list[str]:
106         """Fallback: extract content words when LLM keyword extraction
            ↪ fails."""
107         words = re.findall(r"\b[a-zA-Z]{4,}\b", text.lower())
108         return list(dict.fromkeys(w for w in words if w not in
            ↪ _STOPWORDS))[:3]
109
110     #  Retrieval
111
112     def query(
113         self,
114         message: str,
115         crop: str,
116         top_k: Optional[int] = None,
117         llm_keywords: Optional[list[str]] = None,
118     ) -> list[RAGChunk]:
119         """Hybrid retrieval: 8 semantic + 8 keyword-guided candidates →
            ↪ rerank → top-k.
120
121         Args:
122             message:      The enriched retrieval query (crop + disease + user
                ↪ message).
123             crop:         Crop name | selects the correct ChromaDB
                ↪ collection.
124             top_k:        Override the default final chunk count (default 4).
125             llm_keywords: Pre-extracted keywords from KeywordExtractor. If
                ↪ None or [],
126                           falls back to heuristic term extraction.
127
128         Returns:
129             List of RAGChunk objects, reranked and capped at top_k.
130             Returns [] if the crop collection doesn't exist.
131         """
132         k = top_k if top_k is not None else self.top_k
```

```
        crop_norm = crop.lower().strip()

        if not self.store.collection_exists(crop_norm):
            log.debug("Retriever: no collection for crop '%s' | returning
            ↪  empty.", crop_norm)
            return []

        encoder = self._get_encoder()
        embedding = encoder.encode(message, show_progress_bar=False).tolist()

        # Determine keyword list: prefer LLM-generated, fall back to
        ↪  heuristic
        search_keywords = (
            llm_keywords if llm_keywords
            else self._heuristic_fallback_terms(message)
        )
        source_label = "LLM" if llm_keywords else "heuristic"

        #  1. Semantic search: 10 candidates
        SEMANTIC_N = 10
        semantic_results = self.store.query(
            crop=crop_norm, embedding=embedding, n_results=SEMANTIC_N
        )

        # Accumulate candidates: (text, metadata, distance)
        candidates: list[tuple[str, dict, float]] = []
        seen: set[str] = set()

        def _add(doc: str, meta: dict, dist: float) -> None:
            key = doc[:120]  # dedup key = first 120 chars
            if key not in seen:
                seen.add(key)
                candidates.append((doc, meta, dist))

        n_semantic = 0
        if semantic_results:
            docs  = semantic_results.get("documents", [[]])[0]
            metas = semantic_results.get("metadatas", [[]])[0]
            dists = semantic_results.get("distances", [[]])[0]
            for doc, meta, dist in zip(docs, metas, dists):
                _add(doc, meta, dist)
                n_semantic += 1
```

```
173
174         #  2. Keyword-filtered semantic: up to 10 keyword candidates
175         # Each of the LLM keywords → top results with where_document filter
176         # That's ~10 raw keyword candidates → deduplicated
177         import math
178         KEYWORD_PER_TERM = math.ceil(10 / max(len(search_keywords), 1))
179         n_keyword = 0
180         for term in search_keywords:
181             clean_term = term.replace("_", " ")
182             kw_results = self.store.query_keyword_filtered(
183                 crop=crop_norm, embedding=embedding, term=clean_term,
184                 n_results=KEYWORD_PER_TERM,
185             )
186             if not kw_results:
187                 continue
188             kw_docs  = kw_results.get("documents", [[]])[0]
189             kw_metas = kw_results.get("metadatas", [[]])[0]
190             kw_dists = kw_results.get("distances", [[]])[0]
191             for doc, meta, dist in zip(kw_docs, kw_metas, kw_dists):
192                 _add(doc, meta, dist)
193                 n_keyword += 1
194
195         log.info(
196             "Retriever: %d semantic + %d keyword candidates | keywords=%s
                ↪  (%s) | crop='%s'",
197             n_semantic, n_keyword, search_keywords, source_label, crop_norm,
198         )
199
200         #  3. Rerank by combined score
201         # Threshold: slightly relaxed for keyword candidates (they had a hard
            ↪  filter,
202         # so a higher distance is acceptable if the keyword matches strongly)
203         ranked: list[tuple[float, RAGChunk]] = []
204         for doc, meta, dist in candidates:
205             if dist > self.distance_threshold * 1.2:
206                 log.debug(
207                     "Retriever: dropped '%s' dist=%.3f > threshold×1.2=%.3f",
208                     meta.get("section", "?"), dist, self.distance_threshold *
                        ↪  1.2,
209                 )
210                 continue
211             kw = self._keyword_score(doc, search_keywords)
```

```
            combined = 0.7 * max(0.0, 1.0 - dist) + 0.3 * kw
            log.info(
                "Retriever: '%s' dist=%.3f kw=%.2f combined=%.3f",
                meta.get("section", "?"), dist, kw, combined,
            )
            ranked.append((combined, RAGChunk(
                text=doc,
                source=meta.get("source", "unknown"),
                section=meta.get("section", "unknown"),
                score=dist,
            )))

        ranked.sort(key=lambda x: x[0], reverse=True)
        result = [chunk for _, chunk in ranked[:k]]

        log.info(
            "Retriever: returning %d final chunks (from %d candidates) for
            ↪ crop='%s'",
            len(result), len(candidates), crop_norm,
        )
        return result

    def warm_up(self) -> None:
        """Eagerly load the embedding model in the current thread.

        Called at server startup so the model is ready before the first
        ↪ request.
        """
        try:
            self._get_encoder()
            log.info("RAGRetriever embedding model pre-warmed.")
        except Exception as e:
            log.warning("RAGRetriever warm_up failed: %s", e)
```